STUDY GUIDE

.NET Tech Lead

10-day deep dive with examples

Backend Web API · C# · Entity Framework Core · SQL Server

Azure App Service · Azure Functions · Microservices · Legacy migration

Companion to the 10-day interview roadmap. The roadmap is the script you speak. This guide is the explanation and the worked example behind each topic.

Prepared 30 October 2026. Examples target C# 14 on .NET 10 LTS, supported through 14 November 2028. .NET 8 and .NET 9 reach end of support in November 2026.

How to use this guide with the roadmap

Read one topic until you can explain the example without looking. Then close the page and give the 90-second answer from the roadmap out loud. If you cannot reconstruct the example, you do not own the topic yet. If you can only recite the example, you are not ready for the follow-up. The follow-up is always the same shape: what breaks, who feels it, and what you would roll back.

1. Read the explanation once, slowly, and trace the example with a finger or a pen.
2. Cover the code and write the important five lines from memory.
3. Speak the Say this box in under 90 seconds. Add the failure mode only if they ask.
4. On Day 10, use the rapid-fire miniatures as a check, then return to the full day when a miniature feels thin.

What this guide will not do

It will not invent Azure quotas, prices, or a latency number you should memorize. When a limit matters in the room, say which document or metric you would check. It will not replace your own incident stories. The Day 9 samples are scaffolds. Swap in your system, your number, and your decision.

The running example

Most examples are one system: an Orders API for a multi-tenant commerce backend. A partner can POST an order and retry. A customer can GET the order. A worker sends the confirmation email after the database commit. SQL Server or Azure SQL holds the order, the idempotency key, and the outbox row. The API runs on Azure App Service. The email worker is an Azure Function on the isolated worker model, triggered by Azure Service Bus. The user's token comes from Microsoft Entra ID. The process reaches Azure SQL with a managed identity.

Scale you may assume out loud, and you must label it as an assumption: about 50 requests a second at peak, about 2 million orders a year, p95 under 300 milliseconds for the read, email within a minute. If they change the number, change the design. A million events a second is a different system.

Answer shape, with the example attached

1. First sentence: the decision or the definition.
2. Second beat: the Orders example, in ASP.NET Core, EF Core, SQL, or Azure.
3. Third beat: the tradeoff, with a number if you measured one.
4. Stop. The lead point is the team rule, the review check, or the production scar.

Day	You will be able to teach
1	Memory, LINQ, equality, disposal, and async, each with a snippet you can write.
2	SOLID and the patterns you would keep, plus the ones you would delete.
3	The Web API pipeline, auth, status codes, idempotency, and a PR review.
4	Tracking, N+1, migrations, concurrency, and the moment you drop to SQL.
5	A plan, an index, locking, and the login the app is allowed to use.
6	A host, a slot strategy, identity, and a poison-message path.
7	A boundary, a message, an outbox, and a failure the user can see.
8	A strangler plan a manager can fund, including rollback.
9	Two designs on paper and the shape of a lead story.
10	90-second answers, rapid fire, and the questions you ask them.

Stack to keep stable while you study

- Language and runtime: C# 14 on .NET 10. New work targets the LTS line.
- API: ASP.NET Core. Controllers when the team is large and the surface is wide. Minimal APIs when the service is small and the team has agreed.
- Data: EF Core for the aggregate. SQL for the hot query. SQL Server or Azure SQL.
- Hosting: App Service for the request API. Azure Functions isolated worker for event and schedule work.
- Messaging: Azure Service Bus when the consumer must complete the message under a lock.
- Identity: Entra ID for the user. Managed identity for the process. A separate identity for migrations.

Day 1 — C# a tech lead can defend

Day 1 is the language panel. They are not scoring a definition. They are scoring whether you can connect a language rule to a bug in an API: a copy that diverged, a query that ran twice, a thread that blocked, a dictionary key that moved, a socket that leaked. Every section below is one of those bugs, with the code that causes it and the code you would leave in the pull request.

Class, struct, and record

A class is a reference type. Variables hold a reference, and assignment copies that reference, so two variables can observe the same object. The instance lives on the heap. Use a class when identity matters: an EF Core entity, a service, anything the change tracker must recognize as the same row across a query and an update.

A struct is a value type. Assignment copies the data. A local struct may live on the stack. A struct stored as a field of a class lives inside that object on the heap. A struct in an array of structs lives inline in the array. Copying a large struct copies every field. If one copy is mutated, the other copy does not change, which is a silent bug when the struct was meant to be one order draft.

A record is a class with value-based equality, a compact constructor, and a useful ToString, unless you declare a record struct. Use a record for a DTO, a message, or a result. Use a record struct only when the data is small and equality should compare contents. EF entities stay classes, because the tracker depends on reference identity and because a record's synthesized equality is the wrong identity for a row whose fields change.

Example

```
public sealed class Order          // identity: the row
{
    public int Id { get; set; }
    public string Number { get; set; } = "";
    public decimal Total { get; set; }
}

public readonly record struct Money(decimal Amount, string Currency);

public sealed record OrderCreated(int OrderId, DateTimeOffset OccurredUtc);

// A 200-byte mutable struct passed through the API is a review reject.
// Copies diverge, and each pass copies the payload.
public struct OrderDraft
{
    public string CustomerName;
    public string Line1;
    public decimal Total;
    // ... many more fields
}
```

Money is a few bytes, immutable, and compared by value, so a record struct fits. OrderCreated is a message: two instances with the same order id and timestamp are the same fact, so a record fits. Order is an entity:

two instances with the same Id are the same row only because you say so in the model, and EF must track one instance. OrderDraft is the shape a lead rejects. Pass a record or a class instead, and keep the payload small on purpose.

What breaks

A large mutable struct copied into a method, then mutated, leaves the caller's copy unchanged. A record used as an EF entity surprises the tracker and equality in dictionaries. The sentence 'structs live on the stack and classes live on the heap' is incomplete: a struct field inside a class lives with the object.

Say this

Class when identity matters, record when the value is the identity, struct when the value is small and immutable. I reject a large struct in the API layer and a record used as an EF entity.

ref, out, in, and params

All three of ref, out, and in pass a variable by reference so the method can reach the caller's storage. They differ in who initializes it and whether the callee may write. params packs a variable argument list. On a public Web API these are rarely the right surface, because HTTP has no ref. They still show up in parsers, hot numeric code, and the BCL's Try pattern, so a lead can explain them and then choose a return value.

- ref: the caller assigns the variable first. The method may read and write it.
- out: the caller does not need to assign it. The method must assign it on every normal return path. TryParse is the pattern the BCL taught everyone.
- in: a readonly reference. The method cannot assign the variable. This matters for a large readonly struct, because the call does not copy the struct.
- params: the compiler packs arguments. Historically that was an array allocation. Current compilers can use params collections, including spans in hot paths, so a small call may avoid the array. Do not promise that every params call is allocation-free.

Example

```
public readonly record struct Matrix4x4(float M11, float M12 /* ... 16 floats */);

public static void Scale(in Matrix4x4 matrix, float factor)
{
    // reads matrix.M11 without copying the 16 floats
}

public static bool TryParseOrderId(ReadOnlySpan<char> text, out int id)
    => int.TryParse(text, out id);

// The public shape I would actually ship:
public readonly record struct ParsedOrderId(bool Ok, int Id);

public static ParsedOrderId ParseOrderId(string text)
    => int.TryParse(text, out var id)
```

```
        ? new ParsedOrderId(true, id)
        : new ParsedOrderId(false, 0);
```

Scale takes in so a 64-byte matrix is not copied at the call. TryParseOrderId is honest about failure without an exception, which is right for untrusted input. ParsedOrderId is the shape I want on a public method inside the solution: the result is a value, the caller cannot forget to declare an out variable, and the method signature still makes sense if a future transport is a queue rather than a direct call.

What breaks

A public ref or out parameter makes ownership unclear and cannot cross HTTP. An in parameter on a small struct is noise. A params object[] boxes value types. Forgetting that out must be assigned on every path is a compile error, which is the point of out.

Say this

ref reads and writes the caller's variable, out must be assigned by the callee, in is a readonly reference that avoids copying a large struct. On a Web API I return a result or a tuple. TryParse is the exception the BCL already taught.

Deferred LINQ, IEnumerable, and IQueryable

LINQ to Objects builds an iterator. The filter does not run when you write Where. It runs when you foreach, when you call Count, or when you materialize with ToList. A second enumeration runs the query again. That is deferred execution.

IQueryable<T> holds an expression tree, not a delegate. EF Core walks that tree when you enumerate and tries to produce SQL. Where, OrderBy, and Select composed on IQueryable stay in the tree. The moment you treat the query as IEnumerable<T>, further operators run in memory on whatever the SQL already returned. The boundary rule is: keep IQueryable inside the data method, apply the filters, project, and materialize with ToListAsync or SingleOrDefaultAsync. Return a list, a page, or a DTO. Returning IQueryable from a service lets every caller change the SQL, and it drags the DbContext lifetime out of the method that opened it.

Example

```
// Two round trips. Often what you want: a total and a page.
IQueryable<Order> query = db.Orders.Where(o => o.TenantId == tenantId);
int count = await query.CountAsync(ct);
List<OrderListItem> page = await query
    .OrderByDescending(o => o.CreatedUtc)
    .Take(50)
    .Select(o => new OrderListItem(o.Id, o.Number, o.Total))
    .ToListAsync(ct);

// The bug: assigning to IEnumerable too early.
IEnumerable<Order> rows = db.Orders.Where(o => o.TenantId == tenantId);
List<Order> open = rows.Where(o => o.Status == "Open").ToList();
// SQL returned every order for the tenant. "Open" was filtered in the process.
```

```
// Deferred, so this hits the database twice if orders is IQueryable or a lazy iterator.
int a = orders.Count();
int b = orders.Count();
```

The first block is two SQL statements by design. The second block looks like a filter and is a data leak of cost: the tenant's history comes over the wire so the process can throw most of it away. The third block is why a helper that enumerates a query for logging and then returns it will surprise the next caller. Materialize once, at the edge, into a concrete list.

What breaks

A second foreach on a deferred query repeats the work. A predicate EF cannot translate throws in current EF Core, or, if you forced client evaluation, silently pulls rows. A disposed DbContext throws if a caller enumerates an IQueryable after the request scope ended.

Say this

IQueryable is an expression tree that becomes SQL at enumeration. IEnumerable after that point is memory. I keep the query inside the data method, project, and materialize. I do not return IQueryable from a service.

async, await, and the ways production gets it wrong

An async method returns a Task or a ValueTask that represents the work. The compiler builds a state machine. At an await, if the operation is already complete, the method continues. If it is not, the method yields. In ASP.NET Core the continuation is scheduled on the thread pool. There is no SynchronizationContext to marshal back to, which is why ConfigureAwait(false) is not required in application code the way it was in classic ASP.NET or in UI apps. The request thread is free to serve something else while SQL or HTTP is in flight.

The bugs that still take APIs down are independent of that fact. Sync-over-async calls .Result or .Wait and blocks a thread-pool thread until the task finishes. Under load, the pool has no thread left to run the continuation, and latency falls off a cliff. That is thread-pool starvation. async void is for event handlers. Anywhere else, the caller cannot wait, and an exception bypasses the caller's try. A missing CancellationToken means a disconnected client still holds a SQL command and an HTTP call. ValueTask is for a hot path that often completes synchronously. Await a ValueTask once. Task is the default for public methods.

Example

```
[HttpGet("{id:int}")]
public async Task<ActionResult<OrderDto>> Get(int id, CancellationToken ct)
{
    OrderDto? order = await db.Orders.AsNoTracking()
        .Where(o => o.Id == id && o.TenantId == tenant.Id)
        .Select(o => new OrderDto(o.Id, o.Number, o.Total))
        .SingleOrDefaultAsync(ct);          // pass the token into EF
    return order is null ? NotFound() : Ok(order);
}
```

```
// Pass HttpContext.RequestAborted from the controller, or the CancellationToken
// the framework already bound on the action.

// The bug a review rejects:
OrderDto stuck = db.Orders
    .Select(o => new OrderDto(o.Id, o.Number, o.Total))
    .SingleAsync(ct)
    .Result;

// Also a reject: starting unbounded work.
await Task.WhenAll(orderIds.Select(id => ChargeAsync(id))); // 500 charges at once
```

Get yields at SingleOrDefaultAsync. The thread returns to the pool. When SQL completes, a pool thread resumes the method and writes the result. If the client disconnects, ct fires and EF attempts to cancel the command. The .Result line blocks a pool thread for the whole round trip. The WhenAll line is correct as a language feature and dangerous as a capacity plan: each ChargeAsync holds a connection. Cap the degree of parallelism with Parallel.ForEachAsync or a channel, and set a number you can justify against the SQL pool.

What breaks

ConfigureAwait(false) in a controller does not fix a blocked .Result. async void in a service drops exceptions. Awaiting a ValueTask twice is undefined. Catching an exception and not observing a task faults the process through the unobserved-task path only in older behaviors; on modern .NET you still log it, because a swallowed task is a lost order.

Say this

await releases the thread while IO is in flight. I pass CancellationToken from the request into EF and HttpClient. .Result fails review. I cap outbound parallelism so one request cannot open 500 SQL commands.

Equality, hash codes, and records

ReferenceEquals asks whether two variables point at the same object. Object.Equals on a class does that by default. Value types compare contents. GetHashCode must agree with Equals: if two objects are equal, they must have the same hash code. The hash must stay stable while the object sits in a Dictionary or a HashSet. If you override one, override the other, and implement IEquatable<T> on a value-like type so the generic path does not box.

A record synthesizes Equals, GetHashCode, and equality operators from its members. That is what you want for a message or a DTO. It is the wrong default for an entity whose identity is the primary key and whose other fields change. Anonymous types and tuples also use value equality, which is useful in tests and a bad idea for a mutable cache key.

Example

```
public sealed record Sku(string Value);
```

```
Sku a = new("AB-1");
Sku b = new("AB-1");
bool same = a == b; // true, value equality

var seen = new HashSet<Sku>();
seen.Add(new Sku("AB-1"));
bool hit = seen.Contains(new Sku("AB-1")); // true

// The bug: a mutable class used as a key.
public sealed class CacheKey
{
    public string Tenant { get; set; } = "";
    public int OrderId { get; set; }
    public override int GetHashCode() => HashCode.Combine(Tenant, OrderId);
    public override bool Equals(object? obj)
        => obj is CacheKey k && k.Tenant == Tenant && k.OrderId == OrderId;
}

var key = new CacheKey { Tenant = "acme", OrderId = 10 };
var cache = new Dictionary<CacheKey, OrderDto> { [key] = dto };
key.Tenant = "other";          // bucket changed; lookup misses forever
```

Sku works because nothing can change Value after construction. CacheKey looks correct and fails the moment a caller mutates Tenant. The entry remains in the old bucket. A lead bans mutable dictionary keys and uses a record, a value tuple, or a readonly struct for the key. For an Order entity, equality by Id can be useful in tests, and you still do not drop Order instances into a HashSet keyed by a hash of Total.

What breaks

Equal objects with different hash codes corrupt dictionaries. A hash that changes after insertion is a permanent miss. A record's with-expression returns a new instance, so the old instance is unchanged, which is what you want for a message and surprising if someone thought with mutated the entity.

Say this

Equal objects share a hash code, and the hash stays stable in the collection. Records give me that for contracts. Entities stay classes with key identity. I ban mutable dictionary keys.

Disposal and ownership in ASP.NET Core

IDisposable and IAsyncDisposable mean the type owns a resource that the garbage collector will not release on time: a socket, a file handle, an unmanaged buffer, or another disposable it created. using and await using put the lifetime on the screen. Dispose should be idempotent. The DI container disposes the instances it created: scoped services at the end of the request, singletons at host shutdown. You dispose what you created. You do not dispose what you were given, unless the contract says the recipient takes ownership.

DbContext is registered scoped and disposed by the scope. Disposing it inside a handler, or holding it in a singleton, both break that contract. HttpClient created through IHttpClientFactory is the special case worth naming: disposing the client does not tear down the pooled handler. A new HttpClient per call, with its own

handler, exhausts sockets. One static HttpClient held forever can ignore DNS changes. The factory is the default.

Example

```
await using var connection = new SqlConnection(connectionString);
await connection.OpenAsync(ct);
await using var command = connection.CreateCommand();
command.CommandText = "SELECT Total FROM orders.Orders WHERE Id = @id";
// both dispose at the end of the block, even if the query throws

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("Orders")));
// scoped. The request scope disposes it. Business code does not.

builder.Services.AddHttpClient<TaxClient>(client =>
{
    client.BaseAddress = new Uri(builder.Configuration["Tax:BaseUrl"]!);
    client.Timeout = TimeSpan.FromSeconds(2);
});
```

The connection and command are owned by the method, so await using is correct. The DbContext is owned by the scope, so the handler takes it in the constructor and does not call Dispose. TaxClient is typed and created by the factory. A BackgroundService that needs a DbContext creates a scope per unit of work with IServiceScopeFactory, awaits the work, and disposes the scope. It does not take AppDbContext in its own constructor, because the hosted service is a singleton.

What breaks

Disposing a singleton DbContext on the first request makes every later request fail. Not disposing a SqlConnection you opened yourself leaks pooled connections until the pool blocks. Double disposal must be safe. A using around an instance you do not own disposes it for the next caller in the same scope.

Say this

The creator disposes. DI disposes what it created, at the end of the scope or at shutdown. I use the HTTP client factory. Unclear ownership shows up as a leak under load, not in a single debug session.

Boxing, and when it matters on a Web API

Boxing allocates an object on the heap to hold a copy of a value type, which happens when the value is treated as object or as a non-generic interface it implements. Unboxing casts that object back to the value type and checks the type. One box is noise. A box per integer in a tight loop, a non-generic collection, or a logger that takes object for every argument shows up in a trace as allocations.

Generics exist so List<int> stores ints without boxing. LINQ on a List<int> uses a struct enumerator for the same reason. For a normal CRUD API the time is usually in SQL, not in boxes. A lead asks for a trace before talking about boxing, and then talks about it when the trace says the parser or the formatter is allocating.

Example

```
int n = 42;
object boxed = n;       // allocates
int back = (int)boxed;  // unbox, type check

List<int> ids = [10, 11, 12];          // no box per int
var legacy = new ArrayList { 10, 11 }; // boxes each int

// Interface call on a struct can box unless the call site is constrained.
IComparable comparable = n;

void Log<T>(T value) { /* no box when T is a struct and you do not cast to object */ }
void LogObject(object value) { } // LogObject(n) boxes
```

If an orders list endpoint is slow, open the SQL dependency first. If a high-volume webhook parser shows up as Gen0 allocations in a profile, then look at boxing, at string allocations, and at whether a Span-based parse can stay on the stack. Mentioning boxing in the interview without that sequence sounds like year four.

What breaks

Casting a boxed int to long throws. Boxing in a per-request log of every property allocates on the hot path. Micro-optimizing boxing while a query scans a million rows does not move p95.

Say this

Boxing wraps a value type in an object. Generics avoid it. I ask for a trace. If the time is in SQL, boxing is a distraction. If the time is allocations in a parser, then I remove the boxes.

Shared state, lock, and why lock cannot cross await

Immutable data needs no lock. A request-scoped DbContext is confined to one request, so you do not share it across threads. A counter can use Interlocked. A cache of independent keys can use ConcurrentDictionary. A critical section that must not run twice at once in one process can use lock on a private object. The lock statement is reentrant on the same thread and releases if the block throws. It cannot span an await: Monitor is owned by the thread that entered it, and the continuation may run on another thread. The compiler rejects awaiting inside a lock. Across an await, use SemaphoreSlim.WaitAsync and Release in a finally.

A correct lock is still per process. Two App Service instances have two locks and two memories. A static list mutated from requests is a race even on one instance, and it is a consistency bug the moment you scale to two. Shared business state belongs in SQL, in a distributed cache you have chosen on purpose, or in a queue.

Example

```
public sealed class OrderNumberGenerator
{
    private readonly SemaphoreSlim _gate = new(1, 1);
    private readonly AppDbContext _db;

    public async Task<string> NextAsync(CancellationToken ct)
```

```
    {
        await _gate.WaitAsync(ct);
        try
        {
            int n = await _db.Counters.Select(c => c.Value).SingleAsync(ct);
            // still the wrong design once two instances run: the gate is local
            return $"ORD-{n}";
        }
        finally
        {
            _gate.Release();
        }
    }
}

// In-process and honest:
private int _hits;
public void Hit() => Interlocked.Increment(ref _hits);

// The unique number that survives two instances:
// UNIQUE constraint or a SEQUENCE / identity in SQL, not a static int.
```

The semaphore makes NextAsync safe inside one process and still loses if two instances read the same counter. The fix is a SQL sequence, an identity, or an update that increments under a row lock and returns the new value. Interlocked is the right tool for a process-local metric. lock on this, on a Type, or on a string interned by the runtime can deadlock against code you do not own. Use a private readonly object, or do not use lock at all once an await is involved.

What breaks

Awaiting inside lock does not compile, and a hand-rolled Monitor.Enter that is not released on the same thread deadlocks. A static mutable field looks fine in a single-instance dev box and splits brain when the platform scales out. SemaphoreSlim constructed with (1, 1) and released too often throws. Forgetting finally leaks the gate forever.

Say this

lock is for a synchronous critical section on a private object. Across await I use SemaphoreSlim. A lock does not coordinate two App Service instances. Shared state goes in SQL or a real distributed store.

String immutability

A string cannot be changed. Concatenation allocates a new string and copies both sides. In a loop that builds a line per order, that copying becomes quadratic. StringBuilder mutates a buffer. For a handful of pieces, interpolation is clear and the allocation of the final string is the one you were going to pay anyway. An interpolated string still allocates the result. string.Create is the conscious API when you know the length and want to fill a buffer.

Example

```
// Quadratic copying if lines is large.
string csv = "";
foreach (OrderLine line in lines)
    csv += line.Sku + "," + line.Quantity + "\n";

var builder = new StringBuilder(lines.Count * 32);
foreach (OrderLine line in lines)
    builder.Append(line.Sku).Append(',').Append(line.Quantity).Append('\n');
string once = builder.ToString();

// Fine: one result, a few pieces.
string label = $"{order.Number} ({order.Status})";
```

What breaks

Building a million-row export with += will allocate itself into a timeout. StringBuilder is the wrong tool for a SQL query: concatenate parameters and you have an injection bug. Use parameters.

Say this

Strings do not change. A loop that concatenates copies. StringBuilder, or a single interpolation, is the conscious choice. I do not build SQL by concatenating input.

Fundamentals they still ask

These are short. Each one has a decision, not only a definition. If the panel stops at the definition, give the first sentence and stop.

Abstract class versus interface

An abstract class shares a base implementation and consumes the single class inheritance slot. An interface is a contract, and a type can implement many. Start with an interface for a dependency you want to fake in tests. Use an abstract class when subclasses truly share state and a protected algorithm.

```
public interface ITaxCalculator
{
    decimal Tax(Order order);
}

public abstract class FileImporter
{
    public async Task RunAsync(Stream input, CancellationToken ct)
    {
        IReadOnlyList<RawRow> rows = Parse(input);
        await SaveAsync(rows, ct);
    }
    protected abstract IReadOnlyList<RawRow> Parse(Stream input);
    protected abstract Task SaveAsync(IReadOnlyList<RawRow> rows, CancellationToken ct);
}
```

Say this

Interface for a swappable dependency. Abstract class only when the subclasses share real state and behavior. A base class that exists so two types can be stored in one list is usually an interface.

virtual, override, and new

virtual allows a derived class to replace the method through polymorphism. override is that replacement, and the call uses the runtime type. new hides the base method. A call through a base reference runs the base method. Hiding breaks the expectation that the variable's compile-time type does not change the behavior. Leads almost never accept new on a method.

```
public class Pricing { public virtual decimal Total(Order o) => o.Total; }
public class DiscountPricing : Pricing
{
    public override decimal Total(Order o) => o.Total * 0.9m;
}
public class HiddenPricing : Pricing
{
    public new decimal Total(Order o) => 0m;
}

Pricing a = new DiscountPricing();
Pricing b = new HiddenPricing();
decimal discounted = a.Total(order); // 90 percent
decimal hidden = b.Total(order);     // base price, the new method is hidden
```

Say this

override replaces a virtual method for every caller. new hides it and surprises anyone holding the base type. I do not ship new on methods.

const, readonly, and static

const is a compile-time constant. Callers of another assembly embed the value, so changing a public const is a breaking change until every caller recompiles. readonly is assigned in the constructor or at the declaration and then fixed for that instance. static belongs to the type, one per process, not one per App Service farm.

```
public static class Limits
{
    public const int PageSize = 50;          // baked into callers
    public static readonly TimeSpan Timeout = TimeSpan.FromSeconds(2);
}

public sealed class CreateOrderHandler
{
    private readonly IOrderStore _store;     // set in the constructor, then fixed
    public CreateOrderHandler(IOrderStore store) => _store = store;
}
```

Say this

const is compile-time and a versioning trap in a public library. readonly is fixed after construction. static is per process, which is not per farm.

First, FirstOrDefault, and Single

First returns the first match and throws if the sequence is empty. FirstOrDefault returns the default value, null for a reference type, which is the right signal for a missing optional row if you then branch. Single throws unless there is exactly one match, which is what you want on a primary key: zero rows and two rows are both defects. The async EF methods are FirstAsync, FirstOrDefaultAsync, SingleAsync, and SingleOrDefaultAsync. On a primary key, SingleOrDefaultAsync makes a duplicate fail loudly instead of silently returning one of them.

```
Order? order = await db.Orders.AsNoTracking()
    .SingleOrDefaultAsync(o => o.Id == id && o.TenantId == tenantId, ct);

if (order is null)
    return NotFound();
```

Say this

First is any match. Single is exactly one. On a primary key I use Single or SingleOrDefault so a duplicate cannot hide.

throw versus throw ex

throw; inside a catch rethrows the same exception and keeps the original stack trace. throw ex; resets the stack trace at the new throw, so the log points at your catch block and hides the frame that failed. When you need context, wrap a new exception and pass the original as the inner exception.

```
try
{
    await store.SaveAsync(order, ct);
}
catch (DbUpdateException ex)
{
    logger.LogWarning(ex, "Order {OrderId} failed to save", order.Id);
    throw; // keeps the stack
    // throw ex;              // resets the stack. Do not ship this.
    // throw new InvalidOperationException("save failed", ex); // when you add context
}
```

Say this

throw keeps the stack. throw ex resets it. I wrap only when I am adding a context the caller can use, and I keep the original as InnerException.

IEnumerable, ICollection, and IList

IEnumerable<T> can be streamed and may not have a count. ICollection<T> adds Count and mutation. IList<T> adds indexing. IReadOnlyList<T> is the honest return when the caller may index and must not add. Accept the smallest interface the method needs, so a caller can pass an iterator. Return a concrete list or a read-only view so the caller does not trigger a second enumeration by accident.

```
public static decimal Sum(IEnumerable<OrderLine> lines)
    => lines.Sum(l => l.Quantity * l.UnitPrice);

public IReadOnlyList<OrderListItem> Latest()
    => items; // List<OrderListItem> implements IReadOnlyList<T>
```

Say this

Accept the smallest interface. Return a list or a read-only view. I do not return a lazy sequence that the caller will enumerate twice.

Covariance and contravariance

out on an interface type parameter makes it covariant: the interface only produces T, so IEnumerable<string> may be used as IEnumerable<object>. in makes it contravariant: the interface only consumes T, so an Action<object> may be used where an Action<string> is expected, because a method that accepts any object can accept a string. Invariant is the default, and it is why List<string> is not a List<object>: a list both produces and consumes.

```
IEnumerable<string> skus = new[] { "AB-1" };
IEnumerable<object> asObjects = skus; // covariant, out T

Action<object> write = o => Console.WriteLine(o);
Action<string> writeSku = write;     // contravariant, in T

// List<string> is not a List<object>. Add(new object()) would be a lie.
```

Say this

out is producer, covariant. in is consumer, contravariant. Lists are invariant because they do both. I mention this when a generic interface will not compile, not as a feature tour.

Nullable reference types

string? may be null. string should not. The compiler warns. The CLR does not change: these are annotations. Enable them on new projects so a missing null check is a build warning a review can treat as a comment. A suppression belongs next to a proven invariant, with a reason. Order? from SingleOrDefaultAsync is the annotation doing its job.

```
#nullable enable
public sealed record OrderDto(int Id, string Number, string? Coupon);

public OrderDto Require(OrderDto? dto)
```

```
{
    if (dto is null)
        throw new InvalidOperationException("Order was expected.");
    return dto; // compiler now treats dto as non-null
}
```

Say this

Nullable reference types are a compile-time contract. I enable them so reviews talk about null at build time. A suppression needs a reason.

Partial classes and source generators

The compiler merges partial parts of a type into one class. EF migrations, designer files, and source generators use this so generated code and hand-written code can share a type. Keep domain rules out of the generated part. A regeneration then cannot delete the rule. If you need to add behavior to a generated type, put it in a second partial file that you own.

```
// Generated. Regeneration may replace this file.
public partial class OrderConfiguration : IEntityTypeConfiguration<Order>
{
    public void Configure(EntityTypeBuilder<Order> b) { /* generated mapping */ }
}

// Yours. The generator does not own this file.
public partial class OrderConfiguration
{
    // Prefer a separate configuration class you control entirely.
}
```

Say this

Partial merges files. Generators and migrations need that. Hand-written rules live in a file the generator will not overwrite.

Features worth naming, and the traps

From the last few language versions, a lead can name records, required members, primary constructors, collection expressions, the field keyword in a C# 14 property accessor, and extension members. Name one you have used. A list of features you have not shipped is a weaker answer than one real use.

```
public sealed class PricingOptions
{
    public required decimal Rate { get; init; }

    public decimal RatePercent
    {
        get => field;
        set => field = value is >= 0 and <= 1
            ? value
            : throw new ArgumentOutOfRangeException(nameof(value));
```

```
    }
}

List<int> ids = [order.Id, backup.Id];
```

required means the caller must set Rate in an object initializer. The field keyword is the compiler's backing field, so you can validate a set accessor without declaring a private decimal _rate. Collection expressions are the [...] syntax. None of these change the design of the Orders API. They shorten code you were going to write anyway.

Traps on Day 1

Value types do not always live on the stack. A struct field of a class lives on the heap with the object. ConfigureAwait(false) is the right habit in a library that must not resume on a captured context. ASP.NET Core application code has no sync context to return to. Blocking calls and missing cancellation still hurt. Do not recite C# 14 features you have not used.

Say this

I will name one language feature I shipped and the bug it removed. I will not claim structs are the stack, and I will not claim ConfigureAwait(false) is what saves an ASP.NET Core API. Cancellation and not blocking are what save it.

Day 2 — SOLID and design patterns

A traditional panel wants the name. A product panel wants the smallest design that keeps the next change local. Day 2 trains both. For each pattern, know the problem it removes, the problem it adds, and the day you would delete it. The Orders API is still the example: create an order, price it, tax it, and send a confirmation after commit.

Single responsibility

A type has one reason to change. In a Web API that usually means the controller translates HTTP, a handler runs the use case, and EF mapping stays at the data edge. A class that validates the body, calculates tax, writes SQL, and sends email has four reasons to change, and a test for tax has to boot HTTP.

Example

```
[HttpPost("/orders")]
public async Task<ActionResult<OrderDto>> Post(CreateOrderRequest request, CancellationToken ct)
{
    Result<OrderDto> result = await handler.HandleAsync(request.ToCommand(tenant.Id), ct);
    return result.ToActionResult(); // HTTP only
}

public sealed class CreateOrderHandler
{
    public async Task<Result<OrderDto>> HandleAsync(CreateOrder command, CancellationToken ct)
    {
        decimal tax = taxCalculator.Tax(command);
        Order order = Order.Place(command, tax);
        await store.SaveAsync(order, ct);
        return Result<OrderDto>.Ok(OrderDto.From(order));
    }
}
```

The action does not know the tax rate. The handler does not know the status code. The store does not send email. Email is a later consumer of the outbox, which is a different reason to change. A one-class endpoint is still legitimate in a tiny service. Say that. Judgment is the signal, not a six-project solution for eight classes.

What breaks

A controller that owns business rules cannot be tested without the host, and every new channel (a queue consumer, a retry job) copies the rule. A handler that also formats ProblemDetails has two owners for the error catalog.

Say this

The controller translates HTTP. The handler runs the use case. I collapse those on a tiny endpoint and I say so. I split them when a second caller needs the same rule.

Open/closed

Open for extension means a new shipping provider is a new class behind an interface you already registered. Closed for modification means the checkout handler does not grow a switch. This is not a plugin system. Two branches of an if do not deserve a registry.

Example

```
public interface IShippingRate
{
    string Code { get; }
    decimal Quote(Shipment shipment);
}

builder.Services.AddKeyedScoped<IShippingRate, UpsRate>("ups");
builder.Services.AddKeyedScoped<IShippingRate, DhlRate>("dhl");

public sealed class ShippingQuoter(IServiceProvider sp)
{
    public decimal Quote(string code, Shipment shipment)
        => sp.GetRequiredKeyedService<IShippingRate>(code).Quote(shipment);
}
```

Adding a carrier is a class plus a registration. Checkout does not change. If the business has exactly two carriers and the choice is a configuration flag, a single if in the quoter is easier to operate than keyed services. Extract the interface when the third carrier arrives or when the rule needs its own tests.

What breaks

A registry nobody can see in the handler hides the policy. A switch that is the policy, sitting in one tested method, is often the better design. Open/closed applied too early scatters a simple rule.

Say this

A new provider is a new class when the set actually grows. I do not invent a plugin system for two branches of an if.

Liskov substitution

A subtype honors the contract callers already rely on. If the base method returns a rate, the override returns a rate. An override that throws NotSupportedException for a method on the base type will surprise every caller that was given the base type. Prefer a smaller interface that does not include the method you cannot implement.

Example

```
public interface IReadOrders
{
    Task<OrderDto?> FindAsync(int id, CancellationToken ct);
}
```

```
public interface IWriteOrders
{
    Task SaveAsync(Order order, CancellationToken ct);
}

// A reporting replica implements IReadOrders only.
// It does not implement IWriteOrders and then throw on Save.
```

A read replica that is typed as a full store and throws on Save violates Liskov. Callers cannot tell from the type that Save is a lie. Split the interface. The same issue shows up with a square inheriting a mutable rectangle: setting width changes height, and every rectangle algorithm breaks. In a Web API the practical form is a subtype that strengthens preconditions or throws for a base operation.

What breaks

Tests written against the base type pass with the production subtype and fail with the stub, or the reverse. Support then finds a 500 where the contract promised a result.

Say this

A subtype keeps the promises of the type I substituted. If a method does not make sense, it is not on the interface. Throwing NotSupportedException from an override is the smell.

Interface segregation

Callers depend on the slice they use. IOrderQueries and IOrderCommands, or a single IClock, beat one IOrderService with forty methods. A fat interface forces a fake to stub methods the test does not care about, and it forces a replica, a cache, and a writer to pretend they are the same type.

Example

```
public interface IClock
{
    DateTimeOffset UtcNow { get; }
}

public sealed class SystemClock : IClock
{
    public DateTimeOffset UtcNow => DateTimeOffset.UtcNow;
}

// The handler depends on IClock, not on DateTime.
// A test supplies a clock frozen at 2026-10-30T09:00:00Z.
```

IClock is the smallest interface in the solution and the one that prevents a class of flaky tests. IOrderService with Place, Cancel, Refund, Report, Export, and AdminSearch is the one that grows until every change recompiles every caller. Split by caller, not by a diagram.

What breaks

A god interface becomes a merge conflict. A test double that throws on unused methods hides the fact that production started calling them.

Say this

I depend on the slice I use. A forty-method service interface is a bunch of reasons to change wearing one name.

Dependency inversion

High-level policy depends on an abstraction it owns. The SQL or HTTP adapter implements that abstraction. The handler does not reference a concrete SDK type. The composition root, Program.cs, is the place that knows both sides.

Example

```
public interface IOrderStore
{
    Task<Order?> FindAsync(int id, CancellationToken ct);
    Task SaveAsync(Order order, CancellationToken ct);
}

public sealed class EfOrderStore(AppDbContext db) : IOrderStore
{
    public Task<Order?> FindAsync(int id, CancellationToken ct)
        => db.Orders.SingleOrDefaultAsync(o => o.Id == id, ct);

    public async Task SaveAsync(Order order, CancellationToken ct)
    {
        db.Orders.Update(order);
        await db.SaveChangesAsync(ct);
    }
}

builder.Services.AddScoped<IOrderStore, EfOrderStore>();
builder.Services.AddScoped<CreateOrderHandler>();
```

CreateOrderHandler takes IOrderStore. It does not take AppDbContext unless the team has chosen to keep EF in the application project on purpose. Both choices are defensible. The one that fails is a handler that new's up a SqlConnection, because the policy and the adapter are welded together and the test needs a database for a tax rule.

What breaks

An abstraction that mirrors every DbSet method is not inversion. It is a pass-through. Inversion pays off when the port is the use case's language: Find and Save of an aggregate, not GetAll with an Expression.

Say this

The handler owns the interface. Infrastructure implements it. Program.cs wires them. I do not invert a single method that only calls new.

Singleton, the interview implementation, and the one you ship

Singleton means one instance, with controlled creation. The classic teaching form is a private constructor and a static instance. Static initialization in .NET runs once per process and is safe for that pattern. Lazy<T> with LazyThreadSafetyMode.ExecutionAndPublication is the form you can explain if they ask for lazy creation. Double-checked locking is the interview variant you hope never to maintain.

In ASP.NET Core you register AddSingleton. The container creates one instance for the process. The type must be thread-safe. It must not hold per-request data, and it must not take a scoped service such as DbContext in its constructor. A second App Service instance is a second singleton. Cross-instance uniqueness is a database constraint, not a static field.

Example

```
public sealed class PricingCatalog
{
    public static readonly PricingCatalog Instance = new();
    private PricingCatalog() { }
    public decimal Rate { get; } = 0.2m;
}

// What I actually ship:
builder.Services.AddSingleton<PricingCatalog>();
builder.Services.AddSingleton(TimeProvider.System);

// Illegal: singleton capturing a scoped DbContext.
// builder.Services.AddSingleton<OrderCache>(); // if OrderCache takes AppDbContext
```

PricingCatalog.Instance hides the dependency. Tests cannot replace it without a static reset. AddSingleton makes the same lifetime visible in the composition root, and a test can register a different instance. TimeProvider is the modern way to inject a clock as a singleton without writing a static DateTime wrapper. If the catalog must refresh from SQL, it is not a singleton that captures a context. It is a singleton that creates a scope per refresh, or it is scoped.

What breaks

A handwritten singleton in a web app becomes ambient state. A singleton DbContext is disposed after the first request or shared across threads. A lock inside the singleton still does not coordinate the second instance.

Say this

I can write the static singleton. I ship AddSingleton for a thread-safe type with no request data. One instance per process is not one instance per farm. Uniqueness belongs in SQL.

Factory versus Strategy at checkout

A factory decides how to build an object. A strategy decides how to do a behavior. The names get mixed because both are interfaces with more than one implementation. The question to ask is whether you are choosing a constructor or choosing an algorithm.

Example

```
public interface ITaxCalculator
{
    decimal Tax(OrderDraft draft);
}

public sealed class UsTaxCalculator : ITaxCalculator
{
    public decimal Tax(OrderDraft draft) => Math.Round(draft.Subtotal * 0.07m, 2);
}

public sealed class VatCalculator(decimal rate) : ITaxCalculator
{
    public decimal Tax(OrderDraft draft) => Math.Round(draft.Subtotal * rate, 2);
}

public sealed class TaxCalculatorFactory(IConfiguration config)
{
    public ITaxCalculator ForCountry(string country) => country switch
    {
        "US" => new UsTaxCalculator(),
        "DE" => new VatCalculator(config.GetValue<decimal>("Tax:De")),
        _ => throw new InvalidOperationException($"No tax rule for {country}.")
    };
}
```

ITaxCalculator is the strategy: the handler calls Tax and does not know the rate. TaxCalculatorFactory is the factory: it knows which implementation to build. Keyed DI can replace the factory when the set is fixed at startup. A factory method that only calls new UsTaxCalculator and has one implementation is ceremony. Leave the new at the call site until a second construction path exists. IHttpClientFactory is a factory worth naming because it owns handler lifetimes, which is a real decision, not a wrapper around new.

What breaks

A strategy interface with one implementation and a factory that only forwards new add two types and no test. A factory that contains the tax rule and the construction rule becomes the god class the strategy was meant to avoid.

Say this

Factory builds. Strategy behaves. At checkout the tax rule is a strategy. I extract it when the third country arrives or the rule needs its own tests. Two countries and one if can stay in the handler.

Decorator, middleware, and filters

All three add behavior around something else. They sit at different layers. Middleware sits on the HTTP pipeline and runs for requests that reach it: correlation ids, exception handling to ProblemDetails, security headers, request logging. Filters sit closer to MVC actions: authorization, validation, result shaping. A decorator wraps an application interface, such as IPriceService, and adds caching or timing. The price service does not know. A queue consumer can use the decorator with no HTTP at all.

Example

```
public sealed class TimingPriceService(IPriceService inner, ILogger<TimingPriceService> log) :
IPriceService
{
    public async Task<decimal> PriceAsync(Sku sku, CancellationToken ct)
    {
        var watch = Stopwatch.StartNew();
        decimal price = await inner.PriceAsync(sku, ct);
        log.LogInformation("Priced {Sku} in {Ms} ms", sku.Value, watch.ElapsedMilliseconds);
        return price;
    }
}

// Registration order: the decorator is the outer scoped service.
builder.Services.AddScoped<SqlPriceService>();
builder.Services.AddScoped<IPriceService>(sp =>
    new TimingPriceService(sp.GetRequiredService<SqlPriceService>(),
        sp.GetRequiredService<ILogger<TimingPriceService>>()));
```

Discount logic does not belong in middleware just because the user id was in a header. The handler can see the rule, and a test can call the handler without the host. Middleware that reads the body and writes a business decision becomes invisible in code review. Use middleware when the concern is every request. Use a filter when it is every action, or a named set of actions. Use a decorator when the concern is a port that has more than one caller.

What breaks

A decorator that swallows exceptions hides faults from the exception handler. Middleware registered after the endpoint does not run for that concern. A chain of three decorators that always all run is a pipeline you should write as three lines in the handler if each line is the business rule.

Say this

Middleware is HTTP. Filters are MVC. A decorator is the port, so a queue consumer gets the same cache or timing. Business rules stay in the handler where a test can see them.

Repository and unit of work with EF Core

DbContext is a unit of work. It tracks changes and SaveChanges commits them as one transaction by default. DbSet is already a repository. A second generic IRepository<T> that exposes Get, Add, and a predicate usually hides none of the complexity and blocks Include shape, split queries, projections, and ExecuteUpdate.

A focused port is different. IOrderStore with Find and Save of an aggregate lets the domain stay ignorant of EF when you are serious about that boundary. The implementation is a thin class in the infrastructure project. One SaveChanges per use case is the unit of work. Add an explicit transaction when one use case writes through two contexts or needs a lock hint you can explain.

Example

```
public interface IOrderStore
{
    Task<Order?> FindAsync(int id, int tenantId, CancellationToken ct);
    Task SaveAsync(Order order, CancellationToken ct);
}

// The generic repository I refuse as a standard:
public interface IRepository<T> where T : class
{
    IQueryable<T> Query();
    Task AddAsync(T entity, CancellationToken ct);
}
// Query() returns IQueryable and makes every caller a query author.
```

IOrderStore is a boundary. IRepository<T> is a wrapper around DbSet that teaches the team to hide SQL. In a small service I inject AppDbContext into the handler and skip the port. I say that in the interview. Four projects for eight classes is how a lead loses the team.

What breaks

A generic repository that calls SaveChanges inside Add commits half of the use case. A unit-of-work wrapper around one DbContext adds a type and no behavior. Two SaveChanges calls without a transaction can commit the order and fail the outbox row.

Say this

DbContext is the unit of work. I will defend a thin store per aggregate. I will not defend IRepository<T> as a house style. It makes bad queries harder to see.

CQRS and Mediator on a real API

CQRS means the write path and the read path may differ. On a small API that is two handlers and two queries, still on the same database. On a hot read model it can be a projection updated from events, and the read side lags. Split databases when the read and the write need different scaling or different shapes, not when a form has three fields.

Mediator dispatches a command or query object to one handler. MediatR is the common library. It gives a pipeline for logging, validation, and transactions. The cost is indirection: F12 lands on an interface. Use it when the pipeline behaviors earn their keep. Skip it when the controller can call one application service that is already clear.

Example

```
public sealed record CreateOrder(int TenantId, string Sku, int Quantity) :
IRequest<Result<OrderDto>>;

public sealed class CreateOrderHandler(IOrderStore store, ITaxCalculator tax)
    : IRequestHandler<CreateOrder, Result<OrderDto>>
{
    public async Task<Result<OrderDto>> Handle(CreateOrder request, CancellationToken ct)
    {
        // commands change state. This handler does not call other handlers.
        Order order = Order.Place(request, tax);
        await store.SaveAsync(order, ct);
        return Result<OrderDto>.Ok(OrderDto.From(order));
    }
}

public sealed record GetOrder(int TenantId, int OrderId) : IRequest<OrderDto?>;
// GetOrderHandler projects with AsNoTracking. It does not call SaveChanges.
```

A written rule keeps the mediator honest: commands change state, queries do not, handlers do not call other handlers, and cross-cutting work is a pipeline behavior. Without that rule the mediator becomes a web of hidden calls and the stack trace is noise. A read model in a second table is justified when the list query and the aggregate shape have already fought each other in production.

What breaks

Handlers calling handlers hide the transaction boundary. A query handler that saves 'just a last-viewed stamp' makes GET unsafe. Two databases on day one double the operational load before the domain is stable.

Say this

CQRS is a separate read and write shape, on one database until the shapes diverge. Mediator is a dispatcher I adopt when validation and transaction behaviors earn the indirection. Handlers do not call handlers.

Options: which interface you inject

Bind a configuration section to a class and validate it on start. A bad setting then fails deployment, not the first order. IOptions<T> is a singleton and does not see later changes. IOptionsSnapshot<T> is scoped and sees changes on the next request, which fits a normal Web API. IOptionsMonitor<T> is for a singleton, such as a hosted service, that must observe changes. Inject the one the lifetime requires. Do not pass

IConfiguration into the domain. Secrets come from Key Vault through configuration. The options class exposes only what the feature needs.

Example

```
public sealed class PricingOptions
{
    public decimal Rate { get; init; }
    public string Currency { get; init; } = "USD";
}

builder.Services.AddOptions<PricingOptions>()
    .Bind(builder.Configuration.GetSection("Pricing"))
    .Validate(o => o.Rate is >= 0 and <= 1, "Pricing:Rate must be between 0 and 1.")
    .ValidateOnStart();

public sealed class CreateOrderHandler(IOptionsSnapshot<PricingOptions> pricing)
{
    public decimal Rate => pricing.Value.Rate;
}
```

ValidateOnStart runs the checks when the host starts. A slot swap to a configuration that omits Pricing:Rate fails the new slot before traffic moves, if the health check actually starts the host. IOptionsSnapshot inside a scoped handler sees a reload on the next request. A singleton that captured IOptions<PricingOptions> at construction keeps the old rate, which is sometimes what you want and must be a choice.

What breaks

Reading IConfiguration string keys deep in the domain scatters configuration. A missing ValidateOnStart lets the process boot and crash on the first order. Injecting IOptionsSnapshot into a singleton is a captive dependency.

Say this

I bind a typed options class and ValidateOnStart. IOptions is a fixed snapshot, IOptionsSnapshot is per request, IOptionsMonitor is for singletons that watch changes. Startup validation is a release check.

Result or throw

Throw for defects and for situations the caller cannot usefully handle: a broken invariant, a null the type system promised would exist, an IO failure after retries. Return a Result, or a validation problem, for expected business outcomes: invalid input, insufficient stock, a duplicate idempotency key. Validation becomes HTTP 400 with ProblemDetails. A conflict becomes 409. An exception you did not expect becomes 500 from one exception-handler middleware, with a correlation id. Do not catch Exception at the bottom of every method.

Example

```
public readonly record struct Result<T>(T? Value, string? Error)
{
    public bool Ok => Error is null;
    public static Result<T> Success(T value) => new(value, null);
    public static Result<T> Fail(string error) => new(default, error);
}

public Result<Order> Place(CreateOrder command)
{
    if (command.Quantity <= 0)
        return Result<Order>.Fail("Quantity must be positive."); // expected
    if (string.IsNullOrWhiteSpace(command.Sku))
        return Result<Order>.Fail("Sku is required.");
    return Result<Order>.Success(new Order { /* invariant holds */ });
}

// A bug still throws:
if (store is null)
    throw new InvalidOperationException("IOrderStore was not registered.");
```

Quantity less than one is a client mistake. A null store is a defect in composition. Treating both as Result means the hosting model never sees the defect, and crash telemetry stays quiet. Treating both as exceptions means clients parse 500s for validation. The split is the design.

What breaks

A Result that carries a stack trace for bugs hides failures from the process. An exception for 'coupon does not apply' trains clients to retry a 500. Catching Exception and returning Ok is how a failed save becomes a 200.

Say this

Expected business outcomes return a result and a 4xx. Defects throw, and one middleware turns them into a 500 with a trace id. I do not hide bugs inside Result.

When to add a pattern in review

Ask what change you are protecting, who feels the pain, and what the pattern costs in navigation and tests. If the change is speculative, keep the code in one type. If the change has already happened twice, extract an interface at the seam that varied. Ask how the failure will look in logs. A pattern that adds three layers and no correlation id has made the system harder to operate. Write the decision in the PR or a one-page ADR: context, decision, consequences.

Example

```
# ADR 004 — Pricing stays a module
# Context: one team, one query shape, a proposal to extract a pricing service
# and a generic repository before the release.
```

```
# Decision: IPriceService in this process. No IRepository<T>. No new deployable.
# Consequences: SQL stays visible in one class. A later split is possible
# because callers already depend on IPriceService. We do not operate a second
# database this quarter.
```

The ADR is the interview artifact. Offer to walk one: why you kept a modular monolith, why you chose Service Bus, or why you dropped a generic repository. That story beats a pattern list. The relationship cost belongs in the story too. The senior developer who wanted the service can own the module boundary so the later split is still possible.

What breaks

Patterns added for a resume are permanent. The team pays the navigation cost every week. A verbal decision that is not written gets relitigated in the next PR.

Say this

I add a pattern when the change has happened twice or the seam is already obvious. I write the decision down. Speculative flexibility is a cost.

Pattern catalog, with the refusal

The interview questions above are the ones to go deep on. The rest of the catalog still needs a sentence, a shape, and the day you refuse it. Skim these until the refusal is automatic.

Abstract factory

An abstract factory builds a family of related objects together. In a CRUD API this is rare. A per-tenant set of a blob container plus a topic plus a SQL connection factory can be one, if those objects must match. One product is not a family. Inject the one service.

```
public interface ITenantInfrastructure
{
    IOrderStore Store { get; }
    ServiceBusSender Sender { get; }
}
```

Say this

I use it when several objects must be created as a matched set. One product is just a constructor.

Builder

A builder assembles a complex object step by step. WebApplicationBuilder and a test-data builder are the ones you have used. An object with two constructor parameters does not need a builder. Required members and a constructor are clearer.

```
Order order = new OrderBuilder()
    .WithTenant(10)
```

```
    .WithLine("AB-1", 2, 19.00m)
    .WithLine("CD-2", 1, 5.00m)
    .Build();
```

Say this

Builders earn their place in tests and in host setup. Two parameters is a constructor.

Adapter

An adapter makes an outside API look like your interface. A class that wraps a payment SDK is an adapter. Wrapping a type you own and control is an extra hop. Map at the boundary so the vendor's DTO does not leak into the handler.

```
public sealed class StripePaymentAdapter(StripeClient sdk) : IPaymentCapture
{
    public async Task<CaptureResult> CaptureAsync(Money amount, string key, CancellationToken ct)
    {
        var charge = await sdk.Charges.CreateAsync(amount.Amount, key, ct);
        return new CaptureResult(charge.Id, charge.Status == "succeeded");
    }
}
```

Say this

I adapt types I do not own. The handler sees CaptureResult, not the vendor's charge object.

Facade

A facade is one entry in front of a messy subsystem. An application service that places an order by calling inventory, tax, and the store is a facade if those three calls are an accident of history. The failure mode is a new god class. If the facade grows a rule, move the rule into a type with a name.

```
public async Task<OrderDto> PlaceAsync(CreateOrder command, CancellationToken ct)
{
    await inventory.ReserveAsync(command, ct);
    decimal tax = taxCalculator.Tax(command);
    return await store.SaveAsync(Order.Place(command, tax), ct);
}
```

Say this

A facade hides a messy edge. I delete it when it becomes the only place business rules live without a name.

Proxy

A proxy is a stand-in that controls access to the real object. EF Core lazy-loading proxies and a caching proxy are the examples. The trap is a proxy that hides a query. Touching order.Lines then runs SQL, and a loop

becomes N+1. Prefer an explicit load or a projection unless a specific screen needs lazy loading and the team logs the SQL.

```
public sealed class CachingPriceService(IPriceService inner, IMemoryCache cache) : IPriceService
{
    public Task<decimal> PriceAsync(Sku sku, CancellationToken ct)
        => cache.GetOrCreateAsync(sku.Value, entry =>
        {
            entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(5);
            return inner.PriceAsync(sku, ct);
        })!;
}
```

Say this

A proxy controls access. A cache is a proxy I can explain. An EF lazy proxy that hides N+1 is one I leave off.

Observer

Dependents hear a change. A C# event is an in-process observer. Across processes, a Service Bus message is the durable form. A C# event does not survive a crash and does not reach the second instance. If you need a guaranteed cross-service workflow, use a message with a consumer contract, not an event handler on a singleton.

```
public sealed class OrderPlaced
{
    public event EventHandler<int>? Placed;
    public void Raise(int orderId) => Placed?.Invoke(this, orderId);
}
// Fine inside one process. The email worker still needs the outbox.
```

Say this

In-process, an event. Across services, a message. An event is not a transaction.

Command

A command represents a request as an object so it can be queued, logged, or retried. A MediatR IRequest and a Service Bus message are commands. A command object that is only a method call with extra steps is noise. The object earns its place when it crosses a boundary or a pipeline.

```
public sealed record CreateOrder(int TenantId, string Sku, int Quantity, string IdempotencyKey);
```

Say this

I make a command when the request is a message or a pipeline. I keep a method call when it is a method call.

Chain of responsibility

A request passes along handlers until one handles it. ASP.NET Core middleware and the filter pipeline are the chain you already operate. A chain of three classes that always all run is a pipeline you should write plainly. Use the chain when some handlers may stop the request, as authentication does.

```
app.UseExceptionHandler();
app.UseHttpsRedirection();
app.UseRouting();
app.UseCors();
app.UseAuthentication();
app.UseAuthorization();
app.UseRateLimiter();
app.MapControllers();
```

Say this

The HTTP pipeline is my chain of responsibility. I do not add a second chain of classes that always all run.

Template method

A base class defines the algorithm and subclasses replace steps. A file importer with abstract Parse and Save is the honest example. If the base class forces a step a subclass does not need, prefer composition: pass the steps in. Template method plus a deep inheritance tree is hard to test.

```
public abstract class FileImporter
{
    public async Task RunAsync(Stream input, CancellationToken ct)
    {
        var rows = Parse(input);
        await SaveAsync(rows, ct);
    }
    protected abstract IReadOnlyList<RawRow> Parse(Stream input);
    protected abstract Task SaveAsync(IReadOnlyList<RawRow> rows, CancellationToken ct);
}
```

Say this

Template method when the skeleton is stable and the steps vary. Composition when a subclass would have to stub a step it does not want.

Specification

A specification encapsulates a business predicate so two queries can share it and a test can run it without SQL. A specification engine for one Where clause is ceremony. A testable expression for 'open orders older than 30 days' is the version worth keeping, and it must stay translatable if EF will use it.

```
public static class OrderSpecs
{
    public static Expression<Func<Order, bool>> OpenUnpaid(DateTimeOffset cutoff)
        => o => o.Status == "Open" && o.PaidUtc == null && o.CreatedUtc < cutoff;
```

```
}

await db.Orders.Where(OrderSpecs.OpenUnpaid(cutoff)).ToListAsync(ct);
```

Say this

I extract a predicate when two queries and a test share it. I do not build a specification framework for one filter.

Outbox, saga, circuit breaker, strangler, anti-corruption

These show up again on later days with full examples. The Day 2 answer is the refusal.

- Outbox: the business row and the message row commit together. Refuse publish-then-save and save-then-publish.
- Saga: a long process with compensation. Refuse it when one local SQL transaction would do.
- Circuit breaker: fail fast when a dependency is sick, on an HTTP resilience pipeline. Refuse a breaker on your own database for a single timeout with no policy.
- Strangler fig: one route at a time to the new host. Refuse a nine-month rewrite branch.
- Anti-corruption layer: translate a legacy model at the edge. Refuse sharing the legacy entity project to go faster.

Say this

I can name each of these in one sentence and name the day I would not use it. The full mechanics are Days 6, 7, and 8.

Patterns you have rarely shipped

Have a one-line honest answer. A calm scope statement beats a bluffed UML diagram.

- Prototype: clone to avoid expensive setup. I copy DTOs explicitly. I rarely keep a prototype registry.
- Flyweight: share immutable intrinsic state. String interning is the example I cite. I do not design it into CRUD.
- Bridge: split abstraction from implementation so both can vary. I usually inject an interface and stop.
- Composite: a tree of parts treated as one part. A category tree or a menu. Uncommon on a flat order API.
- State: behavior changes with state. An order status transition belongs in a named method that rejects illegal jumps, not a pile of flags.
- Memento: capture and restore state. I use a persisted workflow row or an audit table.
- Visitor: add an operation over a fixed type hierarchy. Compilers and document trees. I have rarely shipped one.
- Interpreter: a grammar and an evaluator. EF expression trees are the interpreter I use. I do not write a rules language unless the business truly authors rules.

Example of State, the one worth showing

```
public sealed class Order
{
    public string Status { get; private set; } = "Pending";

    public void MarkPaid()
    {
        if (Status != "Pending")
            throw new InvalidOperationException($"Cannot pay from {Status}.");
        Status = "Paid";
    }
}
```

That is a state machine small enough to live on the entity. A table of State objects is justified when each status has different behavior and the if statements have already spread. Say you have shipped the method more often than the pattern.

> **Say this**
> I will say I have rarely shipped Visitor or Interpreter. I will show a status method on the order, which is the state rule the business actually has.

A solution shape you can draw

Four projects is a boundary, not a badge. Use it when the team needs the dependency rule enforced by the compiler. Collapse it when the service is small.

- Api: controllers or endpoints, auth, middleware, the composition root.
- Application: use cases, ports, validators. No EF reference if you are serious about the boundary.
- Domain: entities, value objects, domain rules. No ASP.NET reference.
- Infrastructure: EF Core, HTTP adapters, Service Bus. Implements the ports.
- Tests: domain tests with no host, and a smaller set of API tests with WebApplicationFactory.

```
builder.Services.AddScoped<IOrderStore, EfOrderStore>();
builder.Services.AddScoped<CreateOrderHandler>();
builder.Services.AddOptions<PricingOptions>()
    .Bind(builder.Configuration.GetSection("Pricing"))
    .Validate(o => o.Rate is >= 0 and <= 1, "Pricing:Rate must be between 0 and 1.")
    .ValidateOnStart();
```

The compiler enforces the rule only if Api references Application and Infrastructure, Application references Domain, and Domain references nothing of yours. Infrastructure references Application so it can implement the ports. If every project references every other project, the folders are decorative.

What breaks

Domain referencing EF 'just for attributes' pulls the data stack upward. A test project that only boots the host means a tax rule takes seconds to fail. Four projects for eight classes slows the team more than it protects them.

Say this

I draw Api, Application, Domain, and Infrastructure when the boundary is real. I collapse Application and Domain on a small service and I say so.

Day 3 — ASP.NET Core Web API

Day 3 is the request. A lead can draw the pipeline in order, name the status code before naming the library, and review a bad action without a style argument. The Orders API is still the system: a partner POSTs an order, retries, and later GETs it.

The pipeline for a secured POST

The host builds a pipeline. Order matters. Kestrel accepts the connection. Middleware runs in the order you registered it, and the endpoint runs in the middle. A typical API registers exception handling first so it wraps everything else, then HTTPS redirection, routing, CORS, authentication, authorization, rate limiting, and the endpoints. Authentication runs before authorization because authorization needs an identity. The correlation id belongs near the start so the 500 and the SQL log share one id.

Example

```
app.UseExceptionHandler();          // wraps the rest, writes ProblemDetails
app.UseHttpsRedirection();
app.UseRouting();
app.UseCors();                      // browser callers, after routing, before auth
app.UseAuthentication();            // bearer token -> ClaimsPrincipal, or 401
app.UseAuthorization();             // policy -> 403 if the identity may not
app.UseRateLimiter();
app.MapControllers();

// A correlation id, registered at the top, before the exception handler
// so the handler can log the same id:
app.Use(async (ctx, next) =>
{
    if (!ctx.Request.Headers.TryGetValue("X-Correlation-Id", out var incoming))
        incoming = Guid.NewGuid().ToString("n");
    ctx.TraceIdentifier = incoming.ToString();
    ctx.Response.Headers["X-Correlation-Id"] = ctx.TraceIdentifier;
    await next();
});
```

For a secured POST the useful story is: routing selects the endpoint, CORS answers the browser, authentication builds the principal or fails with 401, authorization runs the policy and returns 403 if the caller is known and not allowed, model binding turns the JSON body into a request type, validation runs, the handler opens the scoped DbContext and awaits SQL, and the formatter writes JSON. If an exception escapes, the exception handler logs it with the correlation id and returns ProblemDetails with status 500. MVC filters wrap the action more tightly than middleware: authorization filters, resource filters, action filters, exception filters, and result filters. Say middleware for the request and filters for the action, and do not mix the two orders up.

What breaks

Authorization before authentication runs with an anonymous user and returns 403 for a missing token, which trains clients to treat 'log in' as 'forbidden'. Exception handling registered late does not see exceptions from earlier middleware. CORS in the wrong place fails the browser and works in Postman, so the team thinks the API is fine.

Say this

Exception handling wraps the pipeline. Authentication then authorization. I can draw that before I talk about patterns. The correlation id is on every log line, including the 500.

Controllers or minimal APIs

Controllers group HTTP actions in a class. Filters, model binding, and a large route surface are the habit they fit. Minimal APIs map a handler directly and stay short. Choose controllers when the team is large and the surface is wide. Choose minimal APIs for a small service with a short list of endpoints. Mixing both in one service without a written reason gives you two ways to forget a cancellation token.

Example

```
// Small service. One file can hold the contract.
orders.MapPost("/orders", async (CreateOrderRequest body, CreateOrderHandler handler,
CancellationToken ct) =>
{
    Result<OrderDto> result = await handler.HandleAsync(body.ToCommand(), ct);
    return result.Ok
        ? Results.Created($"/orders/{result.Value!.Id}", result.Value)
        : Results.ValidationProblem(new Dictionary<string, string[]> { ["body"] = [result.Error!]
});
});

// Large team. The same handler, an MVC action, filters already in use.
[HttpPost("/orders")]
public async Task<ActionResult<OrderDto>> Post(CreateOrderRequest body, CancellationToken ct)
{
    Result<OrderDto> result = await handler.HandleAsync(body.ToCommand(), ct);
    if (!result.Ok)
        return ValidationProblem(detail: result.Error);
    return CreatedAtAction(nameof(Get), new { id = result.Value!.Id }, result.Value);
}
```

Both call the same handler. The HTTP style is a team rule, not a different architecture. I would not rewrite controllers to minimal APIs as a migration goal. I would pick one for new endpoints and leave the other until a slice is already open.

What breaks

A minimal API that closes over a scoped service captured at startup is a captive dependency. A controller base class that hides the DbContext encourages actions to query from HTTP methods. Two styles mean two copies of the error catalog.

Say this

Controllers for a wide surface and a team that already uses filters. Minimal APIs for a small service. The handler is the same either way.

DI lifetimes and a captive dependency

The container tracks how long an object lives. Transient is a new instance every time it is resolved. Scoped is one instance per request scope. Singleton is one instance until the process stops. DbContext is scoped. A singleton must not take a scoped service in its constructor. That is a captive dependency: the scoped instance would live as long as the singleton, either disposed after the first request or reused across requests and threads.

Example

```
builder.Services.AddScoped<AppDbContext>(); // or AddDbContext, which is scoped
builder.Services.AddScoped<CreateOrderHandler>();
builder.Services.AddSingleton<IClock, SystemClock>();

// The bug:
public sealed class OrderCache(AppDbContext db) { } // singleton capturing scoped
builder.Services.AddSingleton<OrderCache>();

// The fix for a hosted service, which is a singleton:
public sealed class OutboxRelay(IServiceScopeFactory scopes) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            await using AsyncServiceScope scope = scopes.CreateAsyncScope();
            var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
            await RelayOnceAsync(db, stoppingToken);
            await Task.Delay(TimeSpan.FromSeconds(2), stoppingToken);
        }
    }
}
```

The safe fixes are: make the consumer scoped, or inject IServiceScopeFactory and create a scope at the moment of work. ValidateScopes in development throws when a singleton takes a scoped service. Turn that check on. The registration lives in the composition root, not in the domain. A scoped service that creates a transient disposable and does not own it clearly is the quieter leak: the container does not dispose transients it did not create as part of the scope unless they were resolved from the scope.

What breaks

A captive DbContext throws 'a second operation was started on this context' under parallel requests, or returns one tenant's entities to another. A BackgroundService that constructor-injects DbContext fails at startup if scope validation is on, and fails in production in a worse way if it is off.

Say this

Transient per resolve, scoped per request, singleton per process. DbContext is scoped. A singleton creates a scope at the moment of work. I can find the captive dependency in a short sample.

Authentication and authorization

Authentication answers who called. Authorization answers whether that caller may do this. For a line-of-business API, Entra ID issues tokens and JwtBearer validates them: issuer, audience, lifetime, and signature. The API does not invent passwords if an identity provider already exists. Coarse access is a policy: can-manage-orders requires a scope or an app role. Fine access is resource-based: the handler loads the order and checks the tenant id. Keep the tenant id in a claim and also filter in EF, so one missed if does not leak rows.

401 means unauthenticated. 403 means authenticated and refused. Returning 404 for every forbidden read hides existence. Do that only on purpose, and document it. The user's token and the app's managed identity are different. The user token decides what the human may do. The managed identity decides what the process may do in Azure SQL. Mixing them into one shared SQL login is how an over-privileged app login appears.

Example

```
builder.Services.AddAuthentication().AddJwtBearer(options =>
{
    options.Authority = builder.Configuration["Entra:Authority"];
    options.Audience = builder.Configuration["Entra:Audience"];
});
builder.Services.AddAuthorizationBuilder()
    .AddPolicy("orders.write", p => p.RequireClaim("roles", "orders.write"));

[Authorize(Policy = "orders.write")]
[HttpPost("/orders")]
public async Task<ActionResult<OrderDto>> Post(CreateOrderRequest body, CancellationToken ct)
{
    int tenantId = int.Parse(User.FindFirstValue("tid")!); // from the token, never the body
    if (body.TenantId is not null)
        return ValidationProblem(detail: "TenantId is not accepted from the body.");
    return await handler.HandleAsync(body.ToCommand(tenantId), ct);
}
```

The policy is the coarse gate. The handler still checks that the order id in a GET belongs to tenantId, and the global query filter is the safety net. A test that signs a token for tenant A and requests tenant B's id is the

control. The filter alone is not the security boundary, because IgnoreQueryFilters and a raw SQL path can bypass it.

What breaks

Trusting TenantId or IsAdmin from the JSON body is over-posting. A valid signature with the wrong audience accepts another app's token. [Authorize] missing on one action is an anonymous write. 403 for a missing token makes the client show 'access denied' instead of a login.

Say this

Entra ID identifies the caller. A policy is the coarse check. The tenant comes from the token, and a test proves tenant A cannot read tenant B. 401 is unknown, 403 is known and refused. The managed identity is a different identity from the user.

Status codes and ProblemDetails

Clients write logic against status codes. An error catalog that changes 409 into 400 because a junior found it easier will break those clients. ProblemDetails, RFC 9457, is the error body: type, title, status, detail, and extensions such as traceId. Validation errors add an errors dictionary. Success codes are a contract too.

Example

```
return result.Status switch
{
    "created" => Created($"/orders/{dto.Id}", dto),             // 201
    "replayed" => Ok(dto),                                      // 200, same key, same body
    "invalid" => ValidationProblem(result.Errors),              // 400
    "conflict" => Conflict(new ProblemDetails                   // 409
    {
        Title = "Conflict",
        Detail = "The order was updated by someone else.",
        Extensions = { ["traceId"] = HttpContext.TraceIdentifier }
    }),
    "forbidden" => Forbid(),                                    // 403
    _ => throw new InvalidOperationException(result.Status)     // 500 via the handler
};

// 202 Accepted: the email is later. Location or a status URL points at the order.
// 204: a delete with nothing to return.
// 401: no usable credential. 404: not in the caller's scope.
// 412: If-Match failed. 429: rate limit, with Retry-After. 503: dependency down.
```

Outcome	Status	Body
Read or update that returns a representation	200	The DTO
Resource created	201	The DTO and a Location header
Accepted, finishes later	202	A status URL
Success with nothing to say	204	Empty
Validation	400	ProblemDetails plus errors
Missing or bad credential	401	ProblemDetails
Credential known, not allowed	403	ProblemDetails or empty from Forbid
Not in the caller's scope	404	ProblemDetails
Duplicate key or concurrency conflict	409	ProblemDetails
Over the rate limit	429	Retry-After
Defect on our side	500	ProblemDetails with traceId, no stack in the body
Dependency down, client may retry	503	ProblemDetails

What breaks

200 with an error string in the JSON trains every client to parse bodies. 500 for a validation error triggers retries and pages. A ProblemDetails body that includes the SQL exception text leaks schema to the caller.

Say this

I publish the error catalog with the API. 401, 403, 404, 409, and 500 mean different things and I do not collapse them. ProblemDetails carries the status and the trace id.

Verbs, and POST /orders under retries

GET is safe. PUT replaces the resource. PATCH updates a subset. DELETE is idempotent: doing it twice leaves the resource gone. POST creates or starts work, and a retry can create a second order unless you add an idempotency key. Mobile networks and gateways are retry machines. If the team cannot explain the second POST, the endpoint is not ready.

Example

```
public sealed class IdempotencyRow
{
    public string Key { get; set; } = "";
    public int TenantId { get; set; }
    public string RequestHash { get; set; } = "";
    public int StatusCode { get; set; }
    public string ResponseBody { get; set; } = "";
}

// In the same transaction as the order:
```

```
// unique (TenantId, Key)
var existing = await db.Idempotency.SingleOrDefaultAsync(
    r => r.TenantId == tenantId && r.Key == key, ct);
if (existing is not null)
{
    if (existing.RequestHash != hash)
        return Conflict(); // same key, different body
    return Stored(existing); // same key, same body: replay the first response
}
db.Orders.Add(order);
db.Idempotency.Add(new IdempotencyRow { Key = key, TenantId = tenantId, RequestHash = hash });
await db.SaveChangesAsync(ct);
```

The client sends an Idempotency-Key header, a GUID per user action, not per HTTP attempt. The server stores the key, a hash of the body, and the response, in the same transaction as the order. A retry with the same key and the same body returns the stored response and does not insert a second order. The same key with a different body is 409. Keep the row for as long as the client retries, often about a day, and say that window out loud as a product choice. A unique constraint on the business key, such as one external reference per tenant, still exists. The idempotency key does not replace a data rule. GET, PUT, and DELETE should already be safe to retry.

What breaks

Storing the key after SaveChanges races two retries into two inserts. Hashing nothing means a client bug that reuses a key for a different order silently replays the first order. A key with no retention fills the table. Trusting the client's generated order id as the only idempotency control fails when two clients pick ids from different sequences and the business key was the email or the cart.

Say this

POST is not naturally idempotent. I store an Idempotency-Key with a hash of the body in the same transaction as the order. Same key and same body replays. Same key and different body is 409. The database unique constraint is still the business rule.

Validation, versioning, and pagination

Validation of shape lives in one place, FluentValidation or data annotations, before the handler. The handler can assume the JSON was well formed and still enforce business rules such as stock. Errors use the field names the client sent. Versioning is a contract. A URL segment is obvious for an enterprise API because gateways, caches, and humans see it. A header version keeps the URL stable and is easier to miss. Pick one, document it, and run an overlap window. Measure calls on the old version before you delete it.

Pagination: limit plus a continuation token or a keyset on (CreatedUtc, Id). Offset is acceptable for a small admin screen and gets expensive on a deep page because SQL reads and discards the skipped rows. Cap the page size. Filtering and sorting are an allow-list, never a raw column name from the query string.

Example

```
public sealed record OrderListQuery(int Limit, DateTimeOffset? CreatedBefore, int? IdBefore);

// GET /v1/orders?limit=50&createdBefore=2026-10-30T00:00:00Z&idBefore=900
var page = await db.Orders.AsNoTracking()
    .Where(o => o.TenantId == tenantId)
    .Where(o => cursor == null || o.CreatedUtc < cursor.Created ||
                (o.CreatedUtc == cursor.Created && o.Id < cursor.Id))
    .OrderByDescending(o => o.CreatedUtc).ThenByDescending(o => o.Id)
    .Take(Math.Clamp(query.Limit, 1, 100))
    .Select(o => new OrderListItem(o.Id, o.Number, o.CreatedUtc, o.Total))
    .ToListAsync(ct);

// Sort allow-list:
string sort = query.Sort == "total" ? "Total" : "CreatedUtc";
```

The keyset matches an index on (TenantId, CreatedUtc, Id). The next page sends the last item's created time and id. An unbounded GET that returns every order is an incident waiting for the first large tenant. Paging, a maximum, and a projection that omits heavy columns are acceptance criteria.

What breaks

ORDER BY on a string from the query is SQL injection if you concatenate it, and a scan if you allow every column. Offset 100000 on a hot table reads 100000 rows to throw them away. Two versions that share a DTO force a breaking change into both. Validation only in the UI means the partner's retry script bypasses it.

Say this

Shape validation is one component. Business rules stay in the handler. I version in the URL for an enterprise API and keep an overlap. Lists are keyset paginated, capped, and sorted from an allow-list.

Timeouts, retries, and a slow dependency

Every outbound call has a timeout shorter than the client's patience. Retry only idempotent operations, with a small jittered backoff. Do not retry a POST that creates a charge unless the idempotency story is solid. A circuit breaker opens when the failure ratio trips and fails fast for a cool-down so the dependency can recover. A concurrency limit stops one request, or one instance, from launching unlimited parallel calls. In .NET this belongs on the typed HTTP client. SQL uses a command timeout and an execution strategy for transient errors such as deadlock 1205 or a brief Azure SQL failover. When the policy is exhausted, return 503 or a controlled error. Log the dependency name and the correlation id.

Example

```
builder.Services.AddHttpClient<TaxClient>(client =>
{
    client.BaseAddress = new Uri(builder.Configuration["Tax:BaseUrl"]!);
    client.Timeout = TimeSpan.FromSeconds(2);
```

```
})
.AddStandardResilienceHandler(options =>
{
    options.Retry.MaxRetryAttempts = 2;          // GET-like tax quote only
    options.AttemptTimeout.Timeout = TimeSpan.FromSeconds(1);
    options.CircuitBreaker.FailureRatio = 0.5;
    options.CircuitBreaker.MinimumThroughput = 20;
});

// The call site still passes CancellationToken.
decimal tax = await taxClient.QuoteAsync(draft, ct);
```

The numbers above are an example policy, not a quota to memorize. Say you would set them from the dependency's latency and the user's budget, and you would review them like code. Retries without a budget multiply load on the thing that is already sick. Failing in about two seconds is kinder than three retries that hold a thread for thirty. Health checks split liveness and readiness. Liveness means the process can run. Readiness includes SQL. If liveness checks SQL, a database blip restarts every instance.

```
builder.Services.AddHealthChecks()
    .AddDbContextCheck<AppDbContext>("sql", tags: ["ready"]);
app.MapHealthChecks("/health/live", new HealthCheckOptions { Predicate = _ => false });
app.MapHealthChecks("/health/ready", new HealthCheckOptions { Predicate = c =>
c.Tags.Contains("ready") });
```

What breaks

A retry on a non-idempotent POST double-charges. A circuit breaker with MinimumThroughput of 1 opens on the first blip and fails the rest of checkout. A liveness probe that calls SQL turns a failover into a restart storm. new HttpClient() per request exhausts ephemeral ports.

Say this

Timeouts shorter than the user's patience, retries only when the call is safe, a breaker so a sick dependency fails fast, and a concurrency cap. Liveness does not include SQL. Readiness does. The policy is reviewed like code.

What you look for in a pull request

Read the contract first: route, verb, status codes, auth policy, and who can see the data. Then cancellation tokens into EF and HttpClient. Then over-posting: the client must not bind IsAdmin, TenantId, or Total as authority. Then sync-over-async, a captive dependency, and a catch that swallows exceptions. Then the query shape. Then logs that include tokens, card numbers, or full personal data. Then a test for the happy path and one failure path that would have cost money or leaked a row. Leave braces to the formatter. A review comment teaches a rule the author can apply next time.

Example: the action, and the comments

```
[HttpPost]
public OrderDto Create(OrderDto input)
```

```
{
    var ctx = new AppDbContext();
    var order = new Order { Id = input.Id, Total = input.Total, IsAdmin = input.IsAdmin };
    ctx.Orders.Add(order);
    ctx.SaveChanges();
    _bus.Publish(new OrderCreated(order.Id));
    return input;
}
```

1. The action is synchronous. SaveChanges blocks a thread-pool thread. It should be async and take a CancellationToken.
2. new AppDbContext() skips DI, so there is no scope, no options, and no disposal. Inject the context or, better, a handler.
3. The body is an entity DTO. IsAdmin and Total are client-supplied authority. Tenant and price come from the token and the server.
4. The client-supplied Id lets a caller choose keys and overwrite. The database assigns the id, or the idempotency key is a separate column.
5. Publish sits after SaveChanges and outside a transaction. A crash between them commits an order and loses the email. Write an outbox row in the same SaveChanges.
6. Returning input echoes the client's IsAdmin flag back as if it were stored truth.
7. There is no [Authorize], no status code choice, and no test. A duplicate POST creates a second order.

The comment to leave is a rule, not a rewrite: we do not bind privilege or price from the body, we pass CancellationToken into every IO call, and a message that must match the order is an outbox row in the same transaction. The author can apply that to the next PR without you.

What breaks

A review that only fixes names ships the data bug. A review that rewrites the PR in the comment trains the team to wait for you. A review that ignores the SQL ships a list endpoint that will time out on the next tenant.

Say this

I review the contract, the auth, the cancellation token, the bound fields, and the query. My comment states the rule. Style belongs to the formatter.

Day 4 — Entity Framework Core

Day 4 is the point where a clean C# method becomes a SQL statement, a lock, and a migration. A lead can say what the change tracker is for, when it is turned off, and when the team drops to SQL on purpose. The Orders API still writes one aggregate and reads a page.

The change tracker, and when you turn it off

DbContext is a unit of work and an identity map. A tracking query snapshots the entity. If you change a property, SaveChanges compares current values to the snapshot and writes an UPDATE for the columns that changed. A second query for the same key in that context returns the tracked instance, not a fresh copy from the database. That is useful while you edit an aggregate. It costs CPU and memory on a list, and a stale tracked instance can hide a change another request committed.

AsNoTracking skips the snapshot. Use it for read endpoints. You do not update those instances later in the same context. AsNoTrackingWithIdentityResolution still returns one instance per key when a graph would otherwise duplicate a parent. Keep one context per request. Do not cache a context. Do not use one context across threads.

Example

```
// Command path: tracking is the feature.
Order order = await db.Orders.SingleAsync(o => o.Id == id && o.TenantId == tenantId, ct);
order.MarkPaid();
await db.SaveChangesAsync(ct);
// UPDATE orders.Orders SET Status = @p, RowVersion = ... WHERE Id = @id AND RowVersion = @original

// Read path: no snapshot.
OrderDto? dto = await db.Orders.AsNoTracking()
    .Where(o => o.Id == id && o.TenantId == tenantId)
    .Select(o => new OrderDto(o.Id, o.Number, o.Status, o.Total))
    .SingleOrDefaultAsync(ct);
```

The command loads one aggregate, mutates it through a method that enforces the status rule, and saves. The read projects four columns and never attaches an Order. A list endpoint that tracks every column of every order pays for change detection it will never use. Project to the DTO.

What breaks

Updating an AsNoTracking instance and calling SaveChanges writes nothing, because the context is not watching it. A context shared across requests throws, or worse, mixes tenants. Attaching a DTO and marking every property modified overwrites columns the client did not send.

Say this

Tracking is for the command, where change detection is the point. Reads project and use AsNoTracking. One context per request, never across threads.

Include, split queries, lazy loading, and projection

Eager loading uses Include. One SQL statement with several collection includes multiplies rows: two lines and three shipments become six rows. AsSplitQuery runs a query per include and avoids that explosion. The split queries are not one atomic read, so a concurrent update can make the pieces disagree. Lazy loading pulls related data on property access and hides N+1. Leave it off unless a specific screen needs it and the team can see the queries in logs. Explicit loading is LoadAsync when you already have the parent and sometimes need the children. Project with Select when the caller needs a DTO. The SQL then returns the columns you named.

Example

```
// Cartesian explosion: lines x shipments in one result set.
var heavy = await db.Orders
    .Include(o => o.Lines)
    .Include(o => o.Shipments)
    .ToListAsync(ct);

// Separate queries. Not one atomic snapshot.
var split = await db.Orders.AsSplitQuery()
    .Include(o => o.Lines)
    .Include(o => o.Shipments)
    .Where(o => o.Id == id)
    .SingleAsync(ct);

// The list does not need the graph.
var page = await db.Orders.AsNoTracking()
    .Where(o => o.TenantId == tenantId && (cursor == null || o.CreatedUtc < cursor))
    .OrderByDescending(o => o.CreatedUtc)
    .Take(limit)
    .Select(o => new OrderListItem(o.Id, o.Number, o.Status, o.Total))
    .ToListAsync(ct);
```

Use the graph when you are about to enforce a rule that needs lines and shipments together, and keep the page size small. Use the projection for the list. Lazy loading would let a serializer touch Lines for each order and issue a query per row while writing JSON. That passes a unit test with an in-memory stand-in and fails in production.

What breaks

Two collection includes on a large page return a result set the API then spends more time reading than the database spent finding. AsSplitQuery under READ COMMITTED can observe a line that was deleted between the parent query and the child query. Serializing entities with cycles and lazy loads is an accidental query storm.

Say this

Include for a small aggregate I am about to edit. AsSplitQuery when two collections would multiply rows, and I accept it is not one atomic read. Lists project. Lazy loading stays off.

Finding an N+1 and fixing it

N+1 is one query for the parents and one query per parent for the children, usually a loop or a lazy load. The API looks like a single method. The dependency list in Application Insights, or the SQL log, shows fifty statements for fifty orders. The fix is one query with a projection, or a controlled include. EF cannot make a non-SARGable predicate fast. Measure before and after on a realistic volume, not on ten rows.

Example

```
// The bug. One query, then one query per order.
List<Order> orders = await db.Orders.Where(o => o.TenantId == tenantId).Take(50).ToListAsync(ct);
foreach (Order order in orders)
{
    order.LineCount = await db.OrderLines.CountAsync(l => l.OrderId == order.Id, ct);
}

// One query. The count is a subquery in the projection.
var page = await db.Orders.AsNoTracking()
    .Where(o => o.TenantId == tenantId)
    .OrderByDescending(o => o.CreatedUtc)
    .Take(50)
    .Select(o => new OrderListItem(
        o.Id,
        o.Number,
        o.Lines.Count(),
        o.Total))
    .ToListAsync(ct);
```

The loop is obvious once you see it and invisible in review if the count hides behind a navigation property getter. A team rule helps: list endpoints project, they do not return entities, and a loop around a query needs a log sample in the PR. One explicit SQL statement is easier to tune than a clean-looking loop.

What breaks

The in-memory EF provider does not run SQL, so the N+1 test stays green. A projection that calls a .NET method EF cannot translate throws at runtime. Fixing the N+1 without an index on TenantId still scans.

Say this

I confirm N+1 in the SQL log: one parent query and a child query per row. I replace it with one projection. I measure on a realistic volume, and I still check the index.

Migrations locally and in production

A developer changes the model and adds a migration. EF generates a C# migration and updates the model snapshot. Review the Up method. EF does not always guess a rename. It may drop a column and add a column, and that is data loss. In CI, generate an idempotent SQL script and read it. In production, a pipeline applies that script with a migration identity that has DDL rights. The application identity does not own the schema.

Calling Database.Migrate at startup races when many instances boot, and a bad migration takes the fleet down during boot. A single migrator job, or a pipeline step before traffic shifts, is the controlled path. Backward-compatible steps come first, because a slot swap runs old bits and new bits against the same database. That is expand/contract: add a nullable column, deploy code that writes both, backfill, then enforce the constraint. A rename in one step breaks the instances still on the old bits.

Example

```
// Expand: add a nullable column. Old code ignores it. New code writes it.
migrationBuilder.AddColumn<string>(
    name: "ExternalRef",
    schema: "orders",
    table: "Orders",
    maxLength: 64,
    nullable: true);

// Later, after backfill and after no instance reads the old shape:
migrationBuilder.AlterColumn<string>(
    name: "ExternalRef",
    schema: "orders",
    table: "Orders",
    nullable: false,
    oldNullable: true);

// Pipeline, not Program.cs:
// dotnet ef migrations script --idempotent -o migrate.sql
// applied by the migration identity before the slot receives traffic
```

The idempotent script records itself in the migrations history table, so a rerun skips applied steps. A human still reads the SQL. A drop of Orders.Total because a property was renamed will pass a build that only compiles. The review of the script is the control. One owner of the schema during an overlap with a legacy app is non-negotiable. Two migration histories on one database is how a Friday deploy drops a column the old app still reads.

What breaks

Migrate() from every instance can deadlock on the history table or apply a half-tested migration on boot. A required column added in the same release as the code that reads it breaks the old slot during swap. A down migration in production is rarely rehearsed. Prefer a forward fix.

Say this

Migrations are source-controlled SQL, applied by a pipeline identity before traffic moves. I expand then contract so old and new instances both run. I do not call Migrate at startup.

Two users update the same order

A concurrency token, usually a rowversion column, detects a lost update. SQL Server updates the rowversion on every write. EF sends the original token in the WHERE clause. If zero rows match, someone else wrote

first, and SaveChanges throws DbUpdateConcurrencyException. The API returns 409. The client reloads. Do not catch the exception and blindly save again. That overwrites the other user's change. A merge of a specific field is explicit code, not a retry loop.

Isolation does not replace the token. Isolation controls locks during one transaction. The lost update happens when two transactions read, think, and write back in sequence, each in its own short transaction. Put the token on aggregates where losing an update costs money or trust: orders, balances, inventory. A lookup table does not need a merge UI.

Example

```
public sealed class Order
{
    public int Id { get; set; }
    public string Status { get; set; } = "Pending";
    public byte[] RowVersion { get; set; } = [];
}

modelBuilder.Entity<Order>(b =>
{
    b.ToTable("Orders", "orders");
    b.Property(o => o.RowVersion).IsRowVersion();
});

try
{
    order.MarkPaid();
    await db.SaveChangesAsync(ct);
}
catch (DbUpdateConcurrencyException)
{
    return Results.Conflict(); // 409, client reloads
}

// The UPDATE EF sends is shaped like:
// UPDATE orders.Orders SET Status = @status
// WHERE Id = @id AND RowVersion = @original
// IF @@ROWCOUNT = 0 -> concurrency exception
```

Keep the write transaction short. Do not hold it open while you call a payment provider. Read the order, call the provider with an idempotency key, then update the order with the token. If the token fails, the charge and the order can disagree, which is why the provider call and the database row need their own reconciliation story, often the outbox or a recorded payment id, not a longer lock.

What breaks

A client that omits the token, and a server that then ignores it, silently last-writes. Retrying SaveChanges after a concurrency exception overwrites the winner. Mapping rowversion as an ordinary byte array without IsRowVersion means EF will not put it in the WHERE clause.

Say this

rowversion goes in the WHERE clause. Zero rows is a 409, not a silent overwrite. Isolation does not stop two read-modify-write transactions. I keep the transaction off the HTTP call to the payment provider.

Transactions and the SQL retry strategy

SaveChanges uses a transaction. Several operations that must commit together, including raw SQL or an outbox insert, share one explicit transaction. Azure SQL can fail transiently: a deadlock, a throttle, a failover. An execution strategy retries those errors. A retry must re-run the whole transaction, so the transaction starts inside the strategy's ExecuteAsync delegate. If you start a user transaction outside that delegate, EF refuses the combination or the retry cannot repeat the work. Make the work idempotent inside the delegate, or a retry can double-insert when the first attempt committed and the connection died before the response.

Example

```
var strategy = db.Database.CreateExecutionStrategy();
await strategy.ExecuteAsync(async () =>
{
    await using var tx = await db.Database.BeginTransactionAsync(ct);
    db.Orders.Add(order);
    db.Outbox.Add(new OutboxMessage
    {
        Id = order.MessageId,
        Type = "OrderCreated",
        Payload = JsonSerializer.Serialize(new { order.Id })
    });
    await db.SaveChangesAsync(ct);
    await tx.CommitAsync(ct);
});

// Enable retries when you configure SQL, for transient errors including 1205.
options.UseSqlServer(connectionString, sql => sql.EnableRetryOnFailure());
```

The order and the outbox row commit together or not at all. The relay publishes later. Email, Service Bus, and the card processor stay outside this delegate. A transaction that waits on HTTP holds locks and exhausts the pool. Clear the change tracker at the start of a retry if you add entities manually, because a failed attempt may have left them in the Added state. EF's strategy does this when you use its execution pattern. Know that a custom loop must do it too.

What breaks

BeginTransaction outside ExecuteAsync throws when retries are enabled. A retry of a non-idempotent insert duplicates the order if the first commit succeeded and the client never saw it. Holding the transaction across an HTTP call turns a slow partner into blocked orders.

Say this

The transaction lives inside the execution strategy so a retry can run it again. The bus and the payment call live outside it. If the message must match the commit, it is an outbox row in the same transaction.

Tenant isolation and soft delete

Both are global query filters. EF adds the predicate to queries. The tenant filter compares TenantId to a value from the current request, usually a scoped accessor populated from a claim. The soft-delete filter excludes rows where IsDeleted is true. Index (TenantId, ...) so the filter can seek. IgnoreQueryFilters is rare, audited, and tested, because a single call bypasses every filter on that entity. If you need to bypass only soft delete, model that path on purpose. Unique indexes include the tenant. A filtered unique index can allow the same external reference again after a soft delete.

Example

```
public sealed class AppDbContext(DbContextOptions<AppDbContext> options, ITenant tenant) :
DbContext(options)
{
    protected override void OnModelCreating(ModelBuilder model)
    {
        model.Entity<Order>().HasQueryFilter(o =>
            o.TenantId == tenant.Id && !o.IsDeleted);
        model.Entity<Order>().HasIndex(o => new { o.TenantId, o.ExternalRef })
            .IsUnique()
            .HasFilter("[IsDeleted] = 0");
    }
}

// Admin repair, reviewed and tested. It bypasses tenant AND soft delete.
var ghost = await db.Orders.IgnoreQueryFilters()
    .SingleAsync(o => o.Id == id, ct);
```

The tenant value comes from the token when the scope is created, never from the body. For a shared database this is logical isolation. A noisy or regulated tenant can move to a separate database, and you should say that operational cost before you offer it. A filter is not the only control. Authorize the route, reject a cross-tenant id, and add a test that a token for tenant A cannot read tenant B's key.

What breaks

A new entity without the filter leaks rows. IgnoreQueryFilters in a list endpoint leaks them back. A unique index that ignores IsDeleted blocks a customer from reusing a reference after a delete. Raw SQL does not get the filter. Write the predicate yourself or call a procedure that has it.

Say this

Tenant and soft delete are query filters, indexed, with a test that tenant A cannot read tenant B. The tenant id comes from the token. IgnoreQueryFilters is an audited escape. The filter is a safety net, not the only authorization check.

ExecuteUpdate, raw SQL, or a stored procedure

Load and save through the tracker when you are enforcing an aggregate's rules and updating a few rows. ExecuteUpdate or ExecuteDelete when you are doing a set operation and do not want to load the rows. They skip the change tracker and also skip some save interceptors, so a tenant stamp or an audit interceptor will not see them. Raw SQL or a procedure when the query is a reporting shape EF translates badly, when you need a lock hint you have justified, or when a DBA-owned procedure is the agreed contract. FromSql and ExecuteSql use parameters. Do not concatenate user input into SQL.

Example

```
// Set-based. One UPDATE. Interceptors on SaveChanges do not run.
int expired = await db.Orders
    .Where(o => o.Status == "Pending" && o.CreatedUtc < cutoff)
    .ExecuteUpdateAsync(s => s.SetProperty(o => o.Status, "Expired"), ct);

// Parameterized. The value is not concatenated.
int id = 42;
var row = await db.Database.SqlQuery<OrderTotal>(
    $"SELECT Id, Total FROM orders.Orders WHERE Id = {id}")
    .SingleAsync(ct);

// A procedure is a boundary: source-controlled, deployed with the migration,
// EXECUTE granted to the runtime role, tested like the C#.
```

The interpolated SQL form above is parameterized by the provider. It is not string concatenation. A stored procedure earns its place for a hot, stable query. It does not earn a place as the default data layer, because the business rule then leaves the C# review. If the rule is in a procedure, the procedure needs the same PR, a test, and an owner. If the rule is in C#, the SQL must still be visible in logs.

What breaks

ExecuteUpdate bypasses a concurrency token unless you put it in the Where. A procedure updated in production and not in source control drifts. String concatenation of a sort column is injection. FromSqlRaw with an interpolated string that is not a FormattableString is the API that does not parameterize. Use the form that takes FormattableString or explicit parameters.

Say this

Tracker for the aggregate. ExecuteUpdate for a set. A procedure for a hot query the team has agreed to own in SQL. Parameters always. The lead decision is where the rule is visible.

Behaviors you warn a team about

These are the onboarding sentences. They belong on one page in the repo so a new joiner does not learn them by causing an incident.

- If a predicate cannot translate, it should throw. Keep non-translatable work after ToListAsync on purpose.
- Several collection includes can explode into a huge payload that looks like a slow query.

- A global filter forgotten on a new entity leaks tenant data or returns soft-deleted rows.
- A migration that drops a column because a property was renamed will compile. Someone reads the SQL.
- Lazy loading in a loop passes an in-memory test and fails against SQL.
- The in-memory provider is not a database. Query filters, transactions, and constraints get an integration test against SQL.
- Startup Migrate is forbidden. The pipeline applies a reviewed script.
- A client-generated entity graph attached with Update marks every column modified and races the concurrency token.

Example

```
// Looks portable. The in-memory provider will not prove the filter or the index.
public class OrderListTests : IClassFixture<SqlFixture>
{
    [Fact]
    public async Task Tenant_a_does_not_see_tenant_b()
    {
        await fixture.SeedAsync(tenantA: 1, tenantB: 2);
        using var scope = fixture.CreateScope(tenantId: 1);
        var db = scope.Get<AppDbContext>();
        int count = await db.Orders.CountAsync();
        Assert.Equal(1, count);
    }
}
```

What breaks

A green unit-test suite with the in-memory provider, a missing filter, and a startup migration is three incidents stacked. The page in the repo is how you stop teaching those by pager.

Say this

I keep a short data-access page: tracking, migrations, tenant filters, and the ban on startup Migrate. Anything with a filter, a transaction, or a constraint gets a test against SQL.

Day 5 — SQL Server, query performance, and database roles

Day 5 is the hour after the API looks fine and the database does not. A lead does not have to be the DBA. A lead has to read a plan, choose an index with a write cost, explain why two sessions blocked, and refuse db_owner for the application login. The examples use one orders schema.

Clustered index, nonclustered index, and key lookup

A clustered index is the table. The leaf pages are the rows, stored in clustering-key order. A table has one clustered index. Every nonclustered index stores its own key plus the clustering key, so a wide clustering key is copied into every secondary index. A narrow, ever-increasing key such as a bigint identity keeps inserts at the end of the leaf. A random uniqueidentifier as the clustered key scatters inserts and fragments the leaf.

A nonclustered index is a separate structure. A seek jumps to a range. A scan reads all or most of the leaf. A scan is not automatically bad on a small table. A seek is not automatically good if it then does a million key lookups. INCLUDE puts extra columns on the nonclustered leaf so the query can be covered and skip the lookup. Every included column slows writes and grows the index, so you cover the hot query, not every column in the table.

Example

```
CREATE TABLE orders.Orders
(
    Id         bigint IDENTITY(1,1) NOT NULL,
    TenantId   int                  NOT NULL,
    CreatedUtc datetime2            NOT NULL,
    Status     nvarchar(32)         NOT NULL,
    Total      decimal(18,2)        NOT NULL,
    CONSTRAINT PK_Orders PRIMARY KEY CLUSTERED (Id)
);

-- Hot list: tenant, newest first, showing status and total.
CREATE NONCLUSTERED INDEX IX_Orders_Tenant_Created
    ON orders.Orders (TenantId, CreatedUtc DESC, Id DESC)
    INCLUDE (Status, Total);

-- A lookup by business number. The clustering key Id rides along for free.
CREATE UNIQUE NONCLUSTERED INDEX UX_Orders_Tenant_Number
    ON orders.Orders (TenantId, Number);
```

SELECT Id, Status, Total FROM orders.Orders WHERE TenantId = 10 AND CreatedUtc < @cursor seeks the nonclustered index and returns from the leaf because Status and Total are included. Drop the INCLUDE and the same query seeks, then key-looks up into the clustered index once per row. A few lookups are noise. Lookups for tens of thousands of rows dominate the plan. SELECT * also defeats the covering index, because the star includes columns that are not in the leaf.

What breaks

An index that makes the report fast can slow the insert path the business cares about more. A random GUID clustered key fragments the table as it grows. Adding an index without a plan on production-like data is a guess.

Say this

Clustered is the row order. Nonclustered is a second path, with the clustering key attached. INCLUDE covers the hot select list. I ask for the actual plan before I add the index, and I name the write I might slow down.

SARGable predicates, seeks, and scans

SARGable means the column stands alone so the optimizer can seek: column equals a value, column is greater than a value, column is in a small list. A function on the column hides it. YEAR(CreatedUtc) = 2026 cannot seek CreatedUtc, because every row's year has to be computed. Put the function on the constant, or write a range. A leading-wildcard LIKE also hides the column. A trailing wildcard can seek.

Example

```
-- Not SARGable. The index on CreatedUtc cannot seek.
SELECT Id FROM orders.Orders WHERE YEAR(CreatedUtc) = 2026;

-- SARGable. A range seek.
SELECT Id
FROM orders.Orders
WHERE CreatedUtc >= '2026-01-01'
  AND CreatedUtc <  '2027-01-01';

-- Not SARGable. The conversion is on the column.
SELECT Id FROM orders.Orders WHERE CONVERT(date, CreatedUtc) = @day;

-- SARGable. The bounds are on the parameter side.
SELECT Id
FROM orders.Orders
WHERE CreatedUtc >= @day
  AND CreatedUtc <  DATEADD(day, 1, @day);
```

EF Core produces the non-SARGable form when you write o => o.CreatedUtc.Year == 2026. Write a range in the LINQ instead. The same rule applies to o.Email.ToLower() == email. A case-insensitive collation already compares without the function, and the function blocks the seek.

What breaks

The query is correct and the index exists, and the plan still scans because the predicate wrapped the column. Scaling the App Service runs the scan more often. It does not make the predicate SARGable.

Say this

I keep the column bare on the left of the predicate. I put the function on the constant. If the plan scans a large table, I look at the predicate before I look at the instance count.

Statistics, Query Store, and a procedure that is slow for one customer

The optimizer estimates cardinality from statistics. A big load that does not update stats produces a bad estimate and a bad plan. Parameter sniffing is the related trap: SQL compiles a plan for the first parameter values it sees and caches that plan. A plan compiled for a tenant with 20 orders is reused for a tenant with 2 million, or the reverse. The API sees one customer as slow and the others as fine.

Work the problem from the statement, the parameters, and the duration. Query Store on Azure SQL keeps plans and runtime stats so you can compare the actual plan with the estimate. Look for a scan where a seek should be, a huge gap between estimated and actual rows, a key-lookup storm, or a spill. Update statistics if a load invalidated them. Fix the query or the index first. A hint, OPTIMIZE FOR, or a plan guide is a pin you can name and a person who will remove it. A local variable copied from the parameter is an old way to hide the sniffed value, and it has its own estimate. Confirm with the plan before you sprinkle hints.

Example

```
CREATE OR ALTER PROCEDURE orders.ListByTenant
    @TenantId int
AS
SELECT Id, Status, Total
FROM orders.Orders
WHERE TenantId = @TenantId;
-- Compiled for tenant 1 (12 rows): a seek plus lookups may be fine.
-- Reused for tenant 40 (1.8 million rows): the same plan thrashes.
-- First evidence is Query Store: one query id, two parameter sets, two durations.
-- Also check blocking. A fast query waiting on a lock looks slow in the API
-- and innocent when you run it alone.
```

The API symptom is a latency chart for one route. The SQL symptom is a plan, a wait, or a lock. Do not scale out the App Service to hide a scan. More instances run the bad query more often and can exhaust the SQL connection or CPU budget.

What breaks

A hint with no owner stays after the data shape changes and becomes the next incident. OPTION (RECOMPILE) on a query that runs 50 times a second spends CPU compiling. A local-variable trick that 'fixed' sniffing also threw away a good estimate for the common tenant.

Say this

I capture the statement, the parameters, and the plan. I fix the SARGable predicate or the index first. I pin a plan only when I can name the plan and the person who will remove the pin. I do not scale the API to hide a scan.

Isolation, RCSI, and when readers block writers

ACID is the transaction contract: atomicity, consistency, isolation, durability. Isolation is the one panels probe. Under locking READ COMMITTED, a reader takes shared locks and a writer takes exclusive locks, so they block each other. READ COMMITTED SNAPSHOT keeps statement-level read committed semantics and serves the previous row version, so a reader does not take the shared locks that block the writer. Writers still block writers on the same row. SNAPSHOT isolation is transaction-level: the transaction sees the database as of its start, and a write can fail with an update conflict if another transaction changed the row.

Both versioning options use a version store. On modern Azure SQL that store is often under accelerated database recovery, not the old tempdb picture, so say you would confirm the setting rather than recite a tempdb myth. RCSI does not make a read-modify-write safe. Two transactions can still read a balance, add one, and write back. That lost update needs a concurrency token or an atomic UPDATE, which is Day 4.

Example

```
-- Confirm before you recommend it. This is a database change, not a connection flag.
SELECT is_read_committed_snapshot_on, snapshot_isolation_state_desc
FROM sys.databases
WHERE name = DB_NAME();

ALTER DATABASE CURRENT SET READ_COMMITTED_SNAPSHOT ON;

-- Still a lost update under RCSI. Both sessions read 10 and both write 11.
-- Fix: UPDATE orders.Orders SET Total = Total + @delta WHERE Id = @id;
-- or a rowversion token so the second writer gets zero rows and the API returns 409.
```

What breaks

Turning RCSI on during an outage without a test plan changes locking behavior for every query. Versioning is not free. RCSI does not replace a unique constraint or a concurrency token. SNAPSHOT can start failing writes with conflicts the application never handled.

Say this

Locking read committed lets readers and writers block. RCSI serves readers from row versions so they do not block writers. Writers still block writers. I confirm the current setting, and I still use a concurrency token for lost updates.

Deadlocks

A deadlock is a cycle of locks. Session A holds row 1 and wants row 2. Session B holds row 2 and wants row 1. SQL detects the cycle and kills one session with error 1205. The victim can retry when the work is idempotent and the EF execution strategy includes 1205. The lasting fix is a consistent lock order, a shorter transaction, and an index that stops a query from locking a wider range than it needs.

Read the deadlock graph: the processes, the resources, the statements. Deadlocks come from an ORM that updates every column and takes locks in an order nobody wrote down, from triggers that lock a second

table, and from two procedures that touch Orders and Payments in opposite orders. A retry that succeeds without a graph is how the same deadlock stays for a year. Put a metric on 1205. A rising retry rate is a design ticket even when the customer eventually gets a 200.

Example

```
-- Session A
BEGIN TRAN;
UPDATE orders.Orders   SET Status = 'Paid' WHERE Id = 10;  -- locks order 10
UPDATE orders.Payments SET Status = 'Paid' WHERE Id = 88;  -- waits on session B

-- Session B, opposite order
BEGIN TRAN;
UPDATE orders.Payments SET Status = 'Paid' WHERE Id = 88;  -- locks payment 88
UPDATE orders.Orders   SET Status = 'Paid' WHERE Id = 10;  -- waits on session A
-- SQL kills one with 1205.

-- The rule: every transaction touches Orders, then Payments, then Inventory.
-- And the transaction does not stay open across an HTTP call to the card processor.
```

What breaks

Retrying a non-idempotent charge doubles the charge. A missing index turns a point update into a scan that locks far more rows and creates cycles you did not design. Holding the transaction while you call a payment API makes the window long enough for the cycle to form.

Say this

1205 is a lock cycle. I retry only if the work is safe to repeat, then I read the graph and remove the cycle with one lock order, shorter transactions, and the index that stops the extra locks.

Keyset pagination versus offset

Offset pagination is ORDER BY plus OFFSET and FETCH. It is simple and fine for a small admin page. For a deep page, SQL reads and discards the skipped rows, so page 400 costs more than page 1. Inserts at the top also shift rows, so the user sees a duplicate or a skip. Keyset pagination remembers the last key the client saw and asks for the next rows beyond that key. The client sends a cursor, not a page number. The index matches the ORDER BY.

Example

```
-- Offset. Fine for a small screen. Deep pages get slower, and the page moves.
SELECT Id, CreatedUtc, Status, Total
FROM orders.Orders
WHERE TenantId = @TenantId
ORDER BY CreatedUtc DESC, Id DESC
OFFSET @Skip ROWS FETCH NEXT @Take ROWS ONLY;

-- Keyset. @CursorUtc and @CursorId are the last row of the previous page.
-- The first page passes a cursor beyond any real row, or omits the predicate.
```

```
SELECT TOP (@Take) Id, CreatedUtc, Status, Total
FROM orders.Orders
WHERE TenantId = @TenantId
  AND (
        CreatedUtc < @CursorUtc
        OR (CreatedUtc = @CursorUtc AND Id < @CursorId)
      )
ORDER BY CreatedUtc DESC, Id DESC;
```

The matching index is (TenantId, CreatedUtc DESC, Id DESC) INCLUDE (Status, Total), the same one from the lookup section. Cap @Take. If an admin UI must jump to page 400, say the cost out loud or filter harder so the page number sits inside a small set. Align the API with the SQL. A cursor in the contract on day one avoids a breaking change later. The Day 3 API section uses the same cursor.

What breaks

An ORDER BY that does not match the index turns the keyset into a sort of a large set. A cursor that omits the tie-breaker Id skips or repeats rows that share a timestamp. An uncapped page size is an incident when one tenant asks for a million rows.

Say this

Offset is for small pages. Keyset seeks from the last key and stays stable. I design the cursor into the API, cap the page, and match the index to the order.

Who the application connects as

SQL Server has server roles and database roles. The tech-lead answer is least privilege for each identity. db_owner can do everything in the database. db_ddladmin can change schema. db_datareader reads every table. db_datawriter modifies every table. Those built-in roles are broad. A lead creates a custom role and grants at schema or object level.

On Azure SQL the API uses a managed identity. You create a user from that identity and add it to the runtime role. There is no SQL password in the connection string. The connection uses Authentication=Active Directory Managed Identity, or the Microsoft.Data.SqlClient equivalent your platform standardizes on. Humans use their own accounts through a group, with auditing. A shared sa password in a wiki is a finding, not a convenience.

Example

```
CREATE ROLE app_orders_runtime AUTHORIZATION dbo;
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::orders TO app_orders_runtime;
GRANT EXECUTE ON SCHEMA::orders TO app_orders_runtime;
ALTER ROLE app_orders_runtime ADD MEMBER [id-api-prod];

CREATE ROLE app_orders_migrator AUTHORIZATION dbo;
GRANT ALTER ON SCHEMA::orders TO app_orders_migrator;
-- Pipeline identity only. Never the App Service.
```

```
CREATE ROLE app_orders_reader AUTHORIZATION dbo;
GRANT SELECT ON SCHEMA::orders TO app_orders_reader;
-- Reporting. No INSERT, no UPDATE.
```

Identity	May do	Must not do
API runtime	DML and EXECUTE on the orders schema	DDL, db_owner, other databases
Function worker	Only the tables and procedures that worker uses	Reuse the API role by habit
Migration pipeline	DDL during release	Be the login the site uses all day
Reporting reader	SELECT on views or a schema	Write
Human admin	Their own audited account	A shared sa password
Local developer	More rights on a dev database	Those rights copied into production

A new table with no grant fails safely when the runtime role is custom. A new table under db_owner fails open: the app can touch audit and payment tables nobody reviewed. Review grants in the migration PR. db_datareader plus db_datawriter is a stepping stone you can explain, and it is broader than you want.

What breaks

Startup Migrate plus db_owner is two problems: many instances racing DDL, and a runtime that can drop tables. A worker that shares the API role can update tables the worker was never meant to touch. A password in app settings will leak into a log or a slot swap.

Say this

The API is a managed identity in a custom runtime role: DML and EXECUTE on its schema, not db_owner. A different identity migrates. If the panel says db_owner is fine because the app migrates at startup, I split both the identity and the job.

Stored procedure or EF query

EF is the default for an aggregate command and an ordinary filter, because the model, the migration, and the C# rule stay in one review. A procedure or a hand-written statement is justified when the plan is critical, the SQL is set-based and ugly in LINQ, or a reporting query is owned by someone who lives in Query Store. The procedure is parameterized, lives in source control, and deploys with the migration. EXECUTE on that procedure, rather than a blanket data-writer role, is the right grant when the procedure is the only write path you want to allow.

Example

```
-- Default: the handler and EF own 'place an order'.
await db.Orders.AddAsync(order, ct);
await db.SaveChangesAsync(ct);

-- Justified: a nightly set-based expire that LINQ would load row by row.
CREATE OR ALTER PROCEDURE orders.ExpireUnpaid
```

```
    @Cutoff datetime2
AS
UPDATE orders.Orders
SET Status = 'Expired'
WHERE Status = 'Pending'
  AND CreatedUtc < @Cutoff;
```

Moving every CRUD action into procedures splits the team and hides the rule from the people who change it. Leaving a hot report in a LINQ query nobody can tune splits the team the other way. Write the decision down so the next developer does not invent a third style.

What breaks

A procedure edited in production and missing from the migration drifts. A business rule that exists only in the procedure is invisible in the C# review. Three data-access styles in one service means nobody owns performance.

Say this

EF for the aggregate. A procedure for a hot, stable, set-based query, in source control, with EXECUTE granted to the runtime role. I write that decision once.

What a unique constraint does that C# cannot

Two API instances can both check that an email is free and both insert. The check in C# is not atomic across processes. A unique index is. Use one for business identity: an idempotency key, an external reference per tenant, one active subscription. Catch the unique violation and return 409. Validate early for a friendly message, and still accept that the database is the authority under concurrency. A filtered unique index enforces uniqueness only for the rows that match the filter, which is what you want when a soft-deleted email should not block a new signup.

Example

```
CREATE UNIQUE INDEX UX_Idempotency
    ON orders.IdempotencyKeys (TenantId, KeyValue);

CREATE UNIQUE INDEX UX_Customer_Email_Active
    ON customers.Customers (TenantId, Email)
    WHERE DeletedUtc IS NULL;

-- In the handler, a duplicate key becomes 409.
// catch (DbUpdateException ex) when (SqlServerException.IsUniqueViolation(ex))
//     return Results.Conflict();
```

What breaks

Check-then-act in the controller creates two charges or two users under retry. A unique index that includes soft-deleted rows blocks a legitimate re-create. Catching every DbUpdateException and returning 409 hides a real bug such as a truncated column.

Say this

If two instances can both pass the if, the rule belongs in a unique index. I still validate early. I return 409 only for the violation I meant to catch.

Waits worth naming

You do not need every wait type. You need to name the one on the screen, ask for the blocker or the plan, and connect it to a decision in the API.

Wait	What it usually means	What you do
CXPACKET / CXCONSUMER	Parallel workers coordinating. Not automatically a problem.	Look for a skewed parallel plan before you disable parallelism.
PAGEIOLATCH	Waiting to read a page. Often a scan or a cold cache.	Find the scan. Do not add API instances.
LCK_M_*	Waiting on a lock.	Find the blocker and the transaction that is still open.
WRITELOG	Waiting on the log flush.	Shorten transactions. Stop chatty commits inside a loop.
RESOURCE_SEMAPHORE	A memory grant is queued. A large sort or hash.	Reduce the sort, or match an index so the sort goes away.
SOS_SCHEDULER_YIELD	CPU is busy.	The query is burning CPU, or the database is. Check the plan's CPU, not only duration.

Example

```
-- Who is blocking whom. The waiter is the symptom. The blocker is the decision.
SELECT r.session_id AS waiter, r.wait_type, r.wait_time,
       r.blocking_session_id AS blocker
FROM sys.dm_exec_requests r
WHERE r.blocking_session_id <> 0;
```

What breaks

Tuning the waiter while the blocker holds a transaction open across an HTTP call fixes nothing. Disabling parallelism because CXPACKET appeared once can make a large report slower.

Say this

I name the wait, then I ask for the blocker or the plan. Lock waits send me to the open transaction. IO and CPU waits send me to the scan, not to a bigger App Service.

Day 6 — Azure App Service and Azure Functions

Day 6 is where the code runs, how a release moves, and how the process proves who it is. The Orders API is the running example: HTTP on App Service, email after commit on a Function, secrets and SQL through managed identity. Do not invent a price, a SKU limit, or a cold-start millisecond. Say which plan property you would check.

Where the code runs

Pick the host from the traffic shape and the failure you can tolerate. The request API wants a process that is already warm. The email worker wants to scale with the queue and must not be able to starve the API. A workflow with waits wants an orchestrator. A platform you do not operate yet is a cost, even when the diagram looks current.

Host	Fits the Orders system when	Watch
App Service	The HTTP API, steady traffic, a latency target	Many instances. Memory is not shared. Slots for release.
Functions, isolated worker	The email, a schedule, a file arrival, short work behind a queue	Cold start on Consumption. A long job needs a plan that stays warm and a timeout that fits.
Durable Functions	Reserve, pay, compensate, with waits	The orchestrator stays deterministic. Work goes in activities.
Container Apps or AKS	You already operate that platform, or you need a sidecar	You are taking on a platform. One Web API does not justify the start.
Hosted BackgroundService	A tiny loop that must live next to the API	It scales with the API, including when the API is what you wanted to protect.

New functions target the isolated worker. Your code runs in a separate process from the Functions host, which is the model that moves forward with .NET 10. The older in-process model shared the host's CLR and is the legacy path. In the room, say isolated worker, say why a cold start exists, and say which plan property matches the latency you need.

> **Say this**
>
> I put the HTTP API on App Service and the email on a queue and a Function. I do not start on Kubernetes for one API, and I do not put a steady user-facing API on a plan that scales to zero unless the product accepts the wait.

App Service or Functions for an order API that also sends email

The HTTP API goes on App Service because the traffic is steady, the read has a p95 target, and the process owns the request. The email goes on a queue. A function triggered by Service Bus sends the mail and completes the message. If the mail provider is slow or down, the API has already committed the order and

returned 201. The function retries through the Service Bus delivery count. The user does not wait on the provider.

A function is a reasonable host for a small HTTP API with spiky traffic when a cold start is acceptable or the plan keeps instances warm. It is a poor host for the whole CRUD API if the product expects hot latency all day. The split is about scaling and failure, not about fashion. The API and the worker scale for different reasons. In one process, a mail-provider outage and a retry loop can starve the threads the API needs.

Example

```
// API: commit the order and the outbox, return 201. Do not SmtpClient.Send here.

// Function, isolated worker. The message is the unit of work.
public sealed class SendOrderEmail(IEmailSender email, IOrderMailLog log)
{
    [Function(nameof(Run))]
    public async Task Run(
        [ServiceBusTrigger("order-confirmed", Connection = "ServiceBus")]
        ServiceBusReceivedMessage message,
        ServiceBusMessageActions actions,
        CancellationToken ct)
    {
        var body = message.Body.ToObjectFromJson<OrderConfirmed>();
        if (await log.AlreadySentAsync(message.MessageId, ct))
        {
            await actions.CompleteMessageAsync(message, ct);
            return;
        }
        await email.SendAsync(body!, ct);
        await log.MarkSentAsync(message.MessageId, ct);
        await actions.CompleteMessageAsync(message, ct);
    }
}
```

What breaks

Sending mail inside POST couples the user's latency to a third party and turns a provider outage into API errors. Completing the message before the send loses the mail on a crash. Sending without a message-id check double-emails on redelivery.

Say this

The API owns the request and returns after the commit. The Function owns the mail and can retry without touching the API thread pool. I split them because they fail and scale differently.

Deployment slots

A staging slot runs the new bits with production-like settings. You warm it, hit the health endpoint, and run a smoke check. Swap exchanges the slots. The previous bits remain on the staging slot for a swap-back. That is

the reversible part of a release, and it is only true when the previous bits still work against the current schema.

App settings and connection strings can be sticky to a slot. Be deliberate. A staging slot must not point at a destructive action by accident, and it must not keep a staging-only secret after swap unless you marked it sticky on purpose. Always On stays on for the API so the process is up. The App Service health check should hit readiness, including a lightweight dependency check, so traffic does not land on an instance that cannot reach SQL. Migrations run as a pipeline step before traffic moves, using the expand/contract rule from Day 4. A swap is not a migration strategy.

Example

```
Release order for a nullable column the new code reads:
1. Pipeline applies the expand script (add nullable column). Migrator identity.
2. Deploy the new bits to the staging slot. Sticky setting: none that point at a
   different database. The slot uses production-like config on purpose.
3. Warm the slot. GET /health/ready must touch SQL and return 200.
4. Swap. Watch the error rate and the SQL dependency time.
5. Swap back if the chart moves and the old bits can still read the schema.

A rename in the same release makes step 5 a lie. The old bits look for the old name.
```

What breaks

A slot swap after a column rename or drop cannot roll back. A sticky connection string that still points at staging after swap sends production traffic at the wrong database. Health that always returns 200 sends traffic at an instance that cannot reach SQL.

Say this

I warm the slot, smoke it, and swap. Swap-back is real only when the previous bits still work on the current schema. Migrations are a pipeline step before traffic moves, not something the slot does on its own.

Secrets and Azure SQL without a password

The App Service has a system-assigned or user-assigned managed identity. That identity is granted access to the Key Vault secrets it needs, and app settings reference those secrets so they appear as configuration. For Azure SQL you create a user for the same identity and add it to app_orders_runtime from Day 5. The SQL client uses an Azure identity credential. There is no SQL password and no secret in source control.

Locally, developers use their own Entra login or a dev database, with a user secret stored outside git. Rotation becomes an identity story. Validate options on startup so a missing vault reference fails the slot before swap, which is the Options rule from Day 2.

Draw three identities every time. The user's token decides what the human may do. The API's managed identity decides what the process may do in SQL, Key Vault, and Service Bus. The pipeline's migration identity applies DDL. Mixing them into one shared SQL login is how an over-privileged app login appears.

Example

```
// App setting, not a password in the repo:
// ConnectionStrings__Orders =
//   Server=tcp:orders.database.windows.net;Database=orders;
//   Authentication=Active Directory Managed Identity;Encrypt=True;

builder.Services.AddDbContext<AppDbContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("Orders")));

// Key Vault reference in an app setting, resolved by the same identity:
// Pricing__Rate = @Microsoft.KeyVault(SecretUri=https://orders-kv.vault.azure.net/secrets/pricing-
rate/)

// SQL user, created once by an admin, not by the app:
// CREATE USER [id-api-prod] FROM EXTERNAL PROVIDER;
// ALTER ROLE app_orders_runtime ADD MEMBER [id-api-prod];
```

What breaks

A connection string with a SQL password in app settings will show up in a diagnostic dump or a careless log. A user-assigned identity that was granted on the slot and not on the swapped app fails at the first query. One identity used for runtime and migrations means the site can run DDL all day.

Say this

Managed identity to Key Vault and to a SQL user in the runtime role. No password in the string. Three identities: the user, the app, and the pipeline. If I cannot name all three, the design is not finished.

Cold start, and which plan you choose

On the Consumption plan, Azure can scale to zero. The next invocation loads the host and the isolated worker. That delay is the cold start. Premium and dedicated plans can keep instances warm, with a higher floor cost. You do not need a memorized millisecond. You need the product question: is a person waiting?

Consumption fits low-volume jobs, a nightly cleanup, and work that can wait. Premium or a dedicated plan fits a user who is waiting, or a function that must hold a connection and run with predictable throughput. Also watch the timeout for the plan. A function that parses a million-row file in one invocation will hit the limit. Split the file into messages, or move that job to a host whose duration matches the work. Day 9's file design does exactly that.

Example

```
Email worker:
  Volume is the queue, not a person staring at a spinner.
  Consumption is acceptable if a cold start still lands the mail inside the
  minute we assumed. If the business says the mail is part of checkout latency,
  buy a plan that keeps an instance warm and write that in the design note.

Order API:
```

```
  People are waiting. App Service, Always On. Not Consumption.

Nightly file of a million rows:
  Not one function invocation. A control message, then chunk messages,
  on a plan whose timeout and concurrency match a chunk.
```

What breaks

Choosing Consumption because the invoice looks small, then discovering checkout waits on a cold start, is a lead miss. A single invocation that reads the whole file dies at the plan timeout and retries from the start.

Say this

Cold start is the cost of scale to zero. If a user is waiting, I keep the instance warm and I put the plan next to the latency target. If the work can finish after the response, Consumption is a real option.

A Service Bus trigger, from delivery to completion

Service Bus delivers under a peek lock. The lock is a lease, not a deletion. The function runs. If it finishes, it completes the message and Service Bus removes it. If it throws, the message is abandoned and becomes visible again after the lock, up to MaxDeliveryCount. After that it is dead-lettered. You inspect the dead-letter reason, fix the poison or the bug, and you have a replay path.

Keep the work shorter than the lock, or renew the lock. A long job whose lock expires is processed twice: another worker receives it. The handler is idempotent either way. A unique key or a processed-message table stops a double charge. Prefetch holds messages on one instance. A huge prefetch on a slow handler pins messages that another instance could have finished.

Example

```
try
{
    await HandleAsync(message, ct);          // writes SQL with a unique MessageId
    await actions.CompleteMessageAsync(message, ct);
}
catch (Exception) // poison or a bug
{
    // Abandon makes it visible again. After MaxDeliveryCount, Service Bus
    // dead-letters it. The rest of the queue stays healthy.
    await actions.AbandonMessageAsync(message);
    throw;
}

// Alert: dead-letter count on order-confirmed > 0.
// Replay: fix the payload or the bug, then resubmit from the dead-letter queue.
// A payload that always throws burns the count and sits there. That is the
// poison message. The happy path is not the design.
```

What breaks

Completing before the SQL commit loses the work. A lock that expires mid-send double-processes. Prefetch of hundreds on a handler that calls a slow provider stalls the queue. No alert on dead-letter depth means the poison sits until a customer asks where the mail is.

Say this

Peek lock, complete after the commit, abandon on failure, dead-letter after the delivery count. The handler dedupes. I alert on dead-letter depth and I can replay. That is the poison-message path.

What belongs in a Durable Functions orchestration

An orchestration coordinates. It can wait, fan out activities, and run compensations. It must be deterministic, because the runtime replays the history. No direct IO, no DateTime.Now, no Guid.NewGuid inside the orchestrator. Those belong in activities. The orchestrator calls them through the context and uses context.CurrentUtcDateTime when it needs a clock. State is checkpointed, so a replay must make the same decisions.

Use Durable Functions when the workflow is long, has a human wait, or needs a compensation owner: reserve inventory, take payment, release inventory if payment fails. Use a single message and a database row when the workflow is one step. Do not hide a simple retry policy inside an orchestration. If the interviewer has not used Durable Functions, you still show that you know when a row and a queue are enough.

Example

```
[Function(nameof(OrderOrchestrator))]
public static async Task Run([OrchestrationTrigger] TaskOrchestrationContext context)
{
    var input = context.GetInput<OrderStart>();
    // Deterministic. No DateTime.Now, no HTTP, no SQL in this method.
    bool reserved = await context.CallActivityAsync<bool>(nameof(Reserve), input);
    if (!reserved)
        return;

    bool paid = await context.CallActivityAsync<bool>(nameof(CapturePayment), input);
    if (!paid)
    {
        await context.CallActivityAsync(nameof(Release), input);
        return;
    }
    await context.CallActivityAsync(nameof(MarkPaid), input);
}
```

What breaks

DateTime.Now or a random id inside the orchestrator changes on replay and the workflow double-charges or takes a different branch. Putting the HTTP call in the orchestrator is the same bug. An orchestration for one email is a platform you now have to debug.

Say this

The orchestrator decides. Activities do the IO. Replay must make the same decision, so the clock and the network stay out of the orchestrator. One step is a queue and a row, not a workflow engine.

What breaks when you scale from one instance to ten

Anything in a static field or an in-memory cache becomes ten copies, possibly inconsistent. A background timer inside the API runs ten times unless you use a distributed lock or move it to one scheduled function. A file on local disk is not shared. Session affinity hides the mistake and then sticks users to a dead instance, so a stateless API turns affinity off.

SQL connections scale with instances. Ten instances with a fat pool can exhaust Azure SQL. Set the pool with the maximum instance count in mind. You do not need a magic pool number in the interview. You need to say the pool times the instance count is the real connection count. A lock or a singleton still covers one process. The second instance has its own lock and its own memory. Shared state belongs in SQL, Service Bus sessions, or a store you chose on purpose.

Example

```
// Correct on one instance. Wrong on ten.
public static class Cache
{
    public static List<Sku> Catalog { get; } = Load();
}

// A timer in the API. Ten instances expire the same orders ten times,
// and ten of them will run during a scale-out you did not schedule.
builder.Services.AddHostedService<ExpireOrdersTimer>();

// The question on every design: what happens on the second instance?
// If the answer is 'we only run one', the next question is what happens
// when the platform restarts that one. Single instance is an availability
// decision, and it belongs on paper.
```

What breaks

A static mutable list corrupts under concurrency and diverges across instances. Affinity left on hides a sticky-session bug until one instance dies and ARR keeps sending people there. An unbounded connection pool times ten is how you tip Azure SQL.

Say this

In-memory state is per process. Timers in the API run once per instance. I turn affinity off for a stateless API, and I size the SQL pool against the maximum instance count.

Production is slow right now

Start from the user symptom and one correlation id, then the request chart: rate, latency, errors, and dependency time. If SQL time dominates, open Query Store and blocking, not a code deploy. If an outbound HTTP dependency dominates, check the timeout, a retry storm, and the circuit breaker. If CPU is high and SQL is fine, look for a hot loop, a bad cache, or thread-pool starvation from blocking calls. Check the deploy marker and the slot-swap time.

Decide rollback versus fix-forward with a clock. If the last deploy lines up and rollback is safe, swap back first and diagnose second. Say a one-line status while you do it. Afterward write the cause, the customer impact, and the guard that would have caught it. The panel listens for sequence. A lead who opens the editor and reads code for twenty minutes while the slot can be swapped back has the wrong order.

Example

```
11:20  GET /orders p95 is above four seconds. Correlation id from one slow request.
11:22  Chart: error rate flat, SQL dependency time up, CPU on the app flat.
11:23  Deploy marker at 11:00. Migration was a nullable column. Old bits are safe.
11:25  Swap back. p95 returns. Then Query Store, not before.
11:40  The new screen's query scans Orders. Covering index in a reviewed script.
       The index ships after the swap, with a plan, not during the first five minutes.
```

What breaks

Scaling out during a lock storm multiplies the waiters. Fixing forward on a schema the old bits cannot read removes the rollback. Debugging for half an hour with no status leaves the stakeholders inventing their own incident.

Say this

Metrics, a reversible action, then root cause. If SQL time moved with the deploy and swap-back is safe, I swap back first. I do not scale the app to hide a scan.

Settings worth naming

- App Service: Always On for the API, HTTPS only, a current TLS policy, a health check path that means ready, ARR affinity off, one identity per app.
- Functions: the plan, the timeout, concurrency per instance, Service Bus MaxDeliveryCount, a dead-letter alert, and a retry that matches the idempotency key.
- Both: structured logs, a correlation id, no secrets in logs, a test that hits the endpoint from outside the region, and a dashboard that splits dependency duration from your own CPU.

```
// Readiness includes the dependency. Liveness does not.
// A database blip must not restart every instance.
builder.Services.AddHealthChecks()
    .AddDbContextCheck<AppDbContext>(tags: new[] { "ready" });

app.MapHealthChecks("/health/live", new HealthCheckOptions
```

```
{
    Predicate = check => !check.Tags.Contains("ready")
});
app.MapHealthChecks("/health/ready", new HealthCheckOptions
{
    Predicate = check => check.Tags.Contains("ready")
});
```

What breaks

Pointing the platform health probe at a check that opens SQL means a brief database failure restarts the fleet. Logging the Authorization header or the connection string turns the dashboard into a leak.

Say this

Always On, affinity off, HTTPS only, ready versus live split. Functions get a plan, a timeout, a delivery count, and a dead-letter alert. Logs have a correlation id and no secrets.

Day 7 — Microservices, as a person who has to operate them

Day 7 is the boundary question. A lead draws a module first, names the data owner, and only then cuts a process. Every extra process adds a network failure, a contract, a dashboard, and an on-call question. The examples stay on Orders, Inventory, and email.

What a service is, and what a folder is

A microservice is a deployable with its own data and a business capability, owned by a team that can release it. A folder or a project inside one solution is a module. A schema with its own login is a step toward a boundary. A separate process that shares tables with another process is a distributed monolith: you paid for the network and kept the coupling.

Start from a modular monolith when one team owns the domain and the boundaries are still moving. Extract a service when a part needs its own scale, its own release train, or a different availability target. Each service writes its own data. Another service gets that data through an API or a message, or it keeps a copy it accepts may lag.

Synchronous HTTP is for a query the user is waiting on, when the caller can tolerate the dependency. Asynchronous messaging is for work that can finish after the response, and for fan-out. Azure Service Bus is for commands and business events that need locks, dead-letter, sessions, and a consumer. Event Grid is for event distribution and for reacting to platform events such as a blob landing. Event Hubs is a high-volume stream, closer to telemetry and ingestion than to one business command that must be completed exactly once from the consumer's point of view.

Example

```
One deployable today:
  Api
    Modules/Orders        writes orders schema, login app_orders_runtime
    Modules/Catalog       writes catalog schema, login app_catalog_runtime
    Modules/Pricing       IPriceService, no database of its own yet

Not a service yet:
  A second App Service that still selects from orders.Orders.

A service, when you cut:
  Catalog API, its own database, its own release.
  Orders hears product changes from a message, or calls Catalog for a read
  the user is waiting on. Orders does not get a SQL login on the catalog tables.
```

What breaks

Two deployables and one shared table means a migration in either service can break the other, and neither dashboard shows the join. Event Hubs for 'order placed' gives you a stream and makes you build lock, dead-letter, and replay yourself. Event Grid for a command that must be completed under a lease is the wrong tool.

Say this

A module is a boundary inside one deployable. A service owns its data and its release. Service Bus for the business message, Event Grid for distribution and platform events, Event Hubs for a stream. A second process on a shared table is a distributed monolith.

When you split, and when you keep the monolith

Keep one deployable while one team is learning the domain, the transactions are local, and you can still draw modules as projects and schemas. Split when a capability has a different scaling profile, a different compliance boundary, or a team that must release without waiting for the rest. Catalog reads often dwarf order writes, and a second team may own catalog. That is a reason. A modern diagram is not a reason.

Before the cut, three things are true: the schema is already independent, a contract test exists, and a team can run the new service next week. If you cannot name the data owner and the failure the user will see, the split is early. In the interview, draw the modules, then one dotted line, and say what has to be true before you cut.

Example

```
Keep together this quarter:
  Orders and pricing. One team. One query shape. IPriceService is the seam.

Cut catalog when:
  Catalog deploys twice a week and orders deploys with a migration window.
  Catalog read p95 is dominated by a workload orders does not share.
  The catalog team can be on call for their own database.

Do not cut payments into a service because a slide had six boxes.
Payments that must commit with the order stay in the order transaction
until you are ready to own a saga.
```

What breaks

A split without a contract test ships a broken JSON field to production as a deploy of 'the other repo'. A split that leaves a shared database recreates the coupling and hides it from the API review.

Say this

I keep a modular monolith until scale, compliance, or a release train forces a cut. The cut needs its own schema, a contract test, and an owner. I can name the failure before I draw the box.

Database per service, and the join you lost

The hard part of a database per service is the report that used to join Orders and Customers. After the split, that join is a distributed query. You have three honest options. The UI calls both services and composes, which fits a small screen. One service stores the few customer fields it needs, updated by events, which fits a hot path and accepts lag. A reporting store ingests both streams, which fits analytics. You do not give the

order service a SQL login on the customer tables. Cross-service work that needs both commits to succeed becomes a saga, not a TransactionScope over two databases.

Example

```
-- Before: one database, one report.
SELECT o.Id, c.Name, o.Total
FROM orders.Orders o
JOIN customers.Customers c ON c.Id = o.CustomerId;

-- After: Orders keeps a copy it is willing to see go stale.
CREATE TABLE orders.CustomerSnapshot
(
    CustomerId int NOT NULL PRIMARY KEY,
    Name nvarchar(200) NOT NULL,
    UpdatedUtc datetime2 NOT NULL
);
-- Filled by CustomerChanged events. The order list does not call Customers
-- on the hot path. A screen that must be current calls Customers at read time
-- and shows a controlled error if Customers is down.

-- The report for finance does not run this join in the order service.
-- It reads a reporting store fed by both streams.
```

What breaks

A SQL login across the boundary becomes the path every future feature uses, and the service boundary is a fiction. A snapshot with no lag budget surprises support when the name on the order is an hour old and nobody wrote that down. A distributed transaction across two databases will fail in exactly the way the saga was invented to avoid, and it will hold locks while it does.

Say this

The service owns its writes. The old join becomes a UI composition, a copied snapshot that may lag, or a reporting store. I say the reporting question in the design review so the shared database does not sneak back.

The outbox, and the crash between save and publish

If you save the order and then publish, a crash or a bus outage between the two leaves a committed order and no message. The email never sends. If you publish and then save, a failed save leaves a message for an order that does not exist. The outbox inserts the order and an outbox row in one SQL transaction. A relay reads unpublished rows, sends them, and marks them published. The relay can send twice if it crashes after the send and before the mark, so the consumer deduplicates on the message id. Polling the outbox is the version you can explain and operate. A transaction-log reader is an optimization you reach for when the poll cannot keep up, and you say that rather than pretending you run one.

Example

```
await using var tx = await db.Database.BeginTransactionAsync(ct);

db.Orders.Add(order);
db.Outbox.Add(new OutboxMessage
{
    Id = messageId,
    Type = "OrderPlaced",
    Payload = JsonSerializer.Serialize(new OrderPlaced(order.Id, order.TenantId)),
    OccurredUtc = time.GetUtcNow()
});
await db.SaveChangesAsync(ct);
await tx.CommitAsync(ct);
// Nothing has been sent. The user's 201 is already honest: the order exists.

// Relay, a separate loop. It may send twice. Consumers dedupe on Id.
var pending = await db.Outbox.Where(m => m.PublishedUtc == null).Take(50).ToListAsync(ct);
foreach (var message in pending)
{
    await sender.SendMessageAsync(new ServiceBusMessage(message.Payload)
    {
        MessageId = message.Id.ToString(),
        Subject = message.Type
    }, ct);
    message.PublishedUtc = time.GetUtcNow();
}
await db.SaveChangesAsync(ct);
```

The outbox table is an operational object. It has a count, an age, and an owner. Alert when the oldest unpublished row is older than the budget you told the business, which in the running example is inside a minute for email. A visible unpublished row is a better failure than a message that existed only in memory.

What breaks

Publish after SaveChanges drops the message on a crash. Publish before SaveChanges emits a ghost order. Marking the row published before the send drops it the other way. A relay with no alert looks like a healthy API and a silent email.

Say this

The order and the outbox row commit together. A relay publishes and may publish twice. Consumers dedupe. I alert on the age of unpublished rows. I never hold the SQL transaction open while I call the bus.

A saga for order, payment, and inventory

A saga is a long-running process with a forward path and a compensation path. There is no distributed lock across the three databases. Orchestration puts the policy in one process manager, which is easier to see, to test, and to change. Choreography lets each service react to events, which couples them less and becomes

harder to trace. For order, payment, and inventory, orchestration is the right default because the compensation policy is a business rule you want in one place.

The order service writes an order in Pending. The orchestrator asks inventory to reserve, then asks payment to capture. If payment fails, it sends a release and marks the order Failed. If the release fails, the saga stays in a compensating state and alerts a human. Each of inventory and payment commits its own local transaction and honors its own idempotency key. A reservation expires if the saga never completes. Timeouts are explicit.

Example

```
States you can query, not only log:
  Pending
  Reserved                inventory said yes, payment not finished
  Paid                    both said yes
  Failed                  payment said no, inventory released
  CompensationFailed      payment said no, release did not stick
  PaymentUnknown          the call timed out; do not capture again blindly

Each call carries the order id as the idempotency key.
Reserve twice is one reservation.
Capture twice is one charge.
A timeout on capture moves to PaymentUnknown and pages a person.
It does not start a second capture because the first one might have landed.
```

What breaks

A second capture after a timeout double-charges. A choreography of three consumers hides the policy that should release inventory. A saga with no stuck-state query means support reads distributed traces to answer 'where is my order'. Compensation that is not idempotent releases twice or not at all.

Say this

I orchestrate this saga because the compensation is one business rule. Each service commits locally and dedupes. I name the stuck states: Pending too long, PaymentUnknown, and CompensationFailed. Those states have a query and an owner.

Idempotent consumption in SQL

At-least-once delivery is the realistic Service Bus contract. Exactly-once processing is your deduplication, not a checkbox on the bus. The message has a stable id. The consumer inserts that id into a processed-message table with a unique primary key, in the same transaction as the business write. A duplicate key means this message was already applied, and the consumer completes it without doing the work again. For a create, the business unique key can be the idempotency key itself. For an increment, you store the message id so you do not add the quantity twice.

Complete the bus message only after the transaction commits. If you complete first and then crash, the work is lost. If you commit first and then crash before complete, the message redelivers and the unique key saves

you. A unit test that calls the handler twice is the cheapest proof. Teams that say the bus is exactly-once and skip the key double-ship one order.

Example

```
CREATE TABLE orders.ProcessedMessages
(
    MessageId uniqueidentifier NOT NULL PRIMARY KEY,
    ProcessedUtc datetime2 NOT NULL
);

BEGIN TRAN;
    INSERT INTO orders.ProcessedMessages (MessageId, ProcessedUtc)
    VALUES (@MessageId, SYSUTCDATETIME());
    -- 2627 or 2601: already applied. Commit nothing else. Complete the message.

    UPDATE orders.Inventory
    SET OnHand = OnHand - @Qty
    WHERE Sku = @Sku;
COMMIT;
-- Complete the Service Bus message only after COMMIT succeeds.
```

What breaks

An increment without the message id adds the quantity on every redelivery. Completing the message before the commit drops the reservation on a crash. A unique key in a different transaction from the stock update still double-applies.

Say this

The bus delivers at least once. I insert the message id in the same transaction as the business write. A duplicate key means I complete and do nothing. I complete the message after commit.

Versioning an event

The event is a contract. It carries a version, a message id, a correlation id, an occurred time, and a payload that tolerates new fields. Adding an optional field is a compatible change. Renaming a field or changing a meaning is a new event type or a new version, and you publish both until consumers move. You do not share a C# entity project across services, because a rebuild then couples their releases. A small contract package or a published schema is enough. Consumers ignore fields they do not understand. They do not throw on an unknown optional field and dead-letter the queue.

Example

```
{
  "specVersion": "1.0",
  "type": "orders.orderPlaced",
  "id": "8f1c...",
  "time": "2026-10-30T12:00:00Z",
  "correlationId": "req-441",
```

```
  "data": { "orderId": 100, "tenantId": 10, "total": 42.00 }
}

A later producer adds "currency": "USD". Old consumers ignore it.
A later producer that renames total to amount publishes
orders.orderPlaced.v2 and keeps v1 until the email worker and inventory move.
The shared project Orders.Entities.dll referenced by both services is the
distributed monolith. Do not do that.
```

What breaks

A required new field dead-letters every old consumer. A shared EF model means a column rename in one service fails the other service's build, or worse, its runtime. Deserializing with a setting that throws on extra members turns a compatible change into an outage.

Say this

Optional fields are compatible. A new meaning is a new version, published beside the old one. I do not share the entity project. Consumers ignore what they do not understand.

Logs, metrics, and traces across services

Every request and every message carries a correlation id. If a message causes another message, carry a causation id too. Logs are structured: correlation id, service name, message id, tenant, and a duration. Do not log the full payload when it contains personal data. Metrics that matter: request rate, latency, error rate, dependency latency, queue depth, dead-letter depth, outbox age, and saga state counts. Traces connect the HTTP call to SQL and to the consumer with OpenTelemetry. Alert on symptoms users feel and on queues that will become those symptoms. One dashboard per service, and one picture of the business flow.

Example

```
HTTP POST /orders
  traceId = 441
  writes orders.Outbox.CorrelationId = 441

Relay
  publishes ServiceBus message ApplicationProperties["traceId"] = 441

Email function and inventory consumer
  start a span with that trace id
  log messageId and the outcome, not the customer's email address

Alert:
  outbox age > the minute we promised
  dead-letter depth > 0
  CompensationFailed count > 0
```

What breaks

Adding correlation ids after five services are live is a project. A dashboard of CPU with no queue depth looks green while mail is an hour behind. Logging the order payload logs a name, an address, and sometimes a payment reference.

Say this

One correlation id crosses HTTP, the outbox, and the consumers. I alert on outbox age and dead-letter depth, not only on CPU. Tracing is in the first release, because retrofitting it is a project.

A downstream service is down

Classify the call when you design the endpoint, not during the incident. If the call is synchronous and required, the user sees a controlled 503 after the timeout and the circuit breaker, and you do not hold threads for a minute. If the work is asynchronous, the user sees 202 Accepted and a status they can poll, and the message waits on the bus or in the outbox. Also decide the backlog: if inventory is down, do you keep accepting orders into Pending, or do you stop at the edge? That is a business decision. Write it down. Automatic retry with backoff and a dead-letter path keeps the rest of the platform moving.

Example

```
Tax service, user is waiting, business says we cannot price without it:
  timeout budget inside the request
  small retry only if the tax call is idempotent
  circuit open -> 503 ProblemDetails, trace id included
  we do not queue a half-priced order

Email, user is not waiting:
  201 as soon as the order and the outbox commit
  the provider outage shows up as outbox age or dead-letters
  the order status stays Confirmed, mail status stays Pending

Inventory down, business has chosen to keep taking orders:
  order stays Pending
  the saga retries reserve
  a query shows Pending older than the reservation window
  support sees the state without reading five logs
```

What breaks

A synchronous tax call with a 60-second timeout and three retries holds the thread pool and amplifies the outage. Accepting orders while inventory is down, with no Pending state, sells stock you cannot reserve. Rejecting orders that could have been queued is a product decision the team invented in the incident channel.

Say this

Required and synchronous becomes a fast 503. Work that can wait becomes 202 and a visible pending state. I decide accept-versus-reject with the product owner before go-live, and I write it down.

The picture you draw from memory

This is the whole day on one page. If you can redraw it and name the failure at each arrow, you can run the design question.

1. The client calls the Order API. The API writes Order and Outbox in one transaction and returns 201.
2. A relay publishes OrderPlaced to Service Bus and then marks the outbox row. A crash in between redelivers. Consumers dedupe.
3. The email function and the inventory consumer both receive. Each inserts the message id with its own write.
4. Inventory failure retries, then dead-letters. The order stays in a state support can query. The email path is independent.
5. The correlation id is on the HTTP log, the outbox row, and both consumers.

```
Client
  |  POST /orders   trace 441
  v
Order API  --commit-->  Orders row + Outbox row
  |  201
  v
Relay  --publish-->  Service Bus order-placed
  |
  +--> Email function      dedupe, send, complete
  +--> Inventory consumer  dedupe, reserve, complete
                              |
                              +--> on poison: dead-letter, order state visible
```

Say this

I draw the commit, the relay, the two consumers, the dedupe, and the dead-letter. If I cannot point at the row support would open, the design is not done.

Day 8 — .NET Framework to modern .NET

Day 8 is a plan a manager can fund. The opening line is the whole day: you do not rewrite the system in a branch for nine months. You inventory the blockers, put a facade in front of the surface that must stay stable, move one vertical slice to .NET 10, run both, and delete the old slice only when the new one has the same logs, the same auth, and a rollback. The goal is an empty Framework footprint on a date, not a heroic cutover weekend.

What actually changes

Modern .NET is cross-platform. The project is SDK-style with PackageReference. Configuration is appsettings, environment variables, and Key Vault. The HTTP pipeline is ASP.NET Core middleware. .NET Framework is Windows-first: System.Web, Global.asax, web.config, sometimes OWIN, and often EF6 or ADO.NET. The .NET Upgrade Assistant and the API analyzer find APIs that do not exist on .NET 10. They do not find a behavioral change or a bad module boundary. The lead still reads the report.

A class library can multi-target net48 and net10 for a bridge period if the code stays on the shared surface. That lets the old host and the new host share a pure domain library. It is a temporary state with an end date. ASP.NET to ASP.NET Core replaces System.Web. HttpContext is a different type. Session, cache, and authentication modules become middleware and services. Web Forms does not come along. MVC views can move with work. Web API controllers move more naturally.

WCF does not ship as a full equivalent. CoreWCF keeps some SOAP services alive on modern .NET. A public contract you can change should become REST or gRPC. Do not promise that every binding will appear. Authentication often moves from Windows auth or OWIN cookies to Entra ID and JWT for an API, or to cookie auth for a site. System.Drawing, COM, the registry, Event Log assumptions, Windows services, and impersonation are the usual blockers. List them in week one.

Example

```
<!-- Bridge library. An end date is part of the PR. -->
<PropertyGroup>
  <TargetFrameworks>net48;net10.0</TargetFrameworks>
  <Nullable>enable</Nullable>
</PropertyGroup>

// Framework code that will not compile on the new host, and must not be
// copied forward:
var user = HttpContext.Current.User;
var setting = ConfigurationManager.AppSettings["TaxUrl"];
```

What breaks

A multi-target library with no end date becomes the permanent compatibility swamp. Treating the analyzer's green report as a behavioral guarantee misses a changed EF query plan and a missing HTTP module. Promising a WCF binding you have not checked becomes the date slip.

Say this

SDK-style projects, ASP.NET Core, and a temporary multi-target library with an end date. The analyzer finds missing APIs. I still inventory modules, WCF, EF, and Windows-only calls myself.

The migration plan

Week zero is an inventory: projects, target framework, NuGet packages, System.Web usage, WCF, EF6 or EDMX, auth type, Windows-only APIs, scheduled jobs, and how you deploy. You run the upgrade analyzer and you sit with the person who knows the ugly paths. You pick .NET 10 as the target. You freeze new features that deepen Framework-only APIs. You add characterization tests around the slices you will move, at the HTTP boundary if unit tests do not exist.

You move a pure class library first, then one vertical API slice, behind the existing URL via a reverse proxy. You ship that slice to a small share of production traffic with the old path still available. You repeat. Schema changes use expand/contract so both runtimes can work. You remove a Framework project only when nothing calls it. You report percent of traffic on the new runtime, not percent of projects converted. Seven of ten class libraries converted is not 70 percent migrated if the revenue path is still on Framework.

Example

```
YARP in front of the existing host. One route moves. The rest stay.

POST /orders     -> new .NET 10 app      (the first slice)
GET  /orders/*   -> new .NET 10 app      (after the write is proven)
GET  /catalog/*  -> old Framework app    (not this quarter)
POST /payments/* -> old Framework app    (until its own slice)

Metric on the steering slide:
  percent of order traffic served by net10
  not: percent of csproj files retargeted

First slice teaches the real cost of auth, EF, and the deploy.
The estimate for the rest comes after that slice is in production.
```

What breaks

A status deck that counts projects instead of traffic hides the revenue path. A feature freeze that is not real means new Framework-only code lands while you migrate. A first slice that is 'the whole solution' teaches you nothing until the ninth month.

Say this

Inventory, one vertical slice behind the current URL, both runtimes live, delete the old slice when traffic, logs, and rollback are proven. I report traffic percent. A single number for eighty projects is fiction.

What usually blocks a Web API

HTTP modules and handlers quietly do auth, logging, or tenant resolution. They become middleware, and a missed one means the new host is missing a control. WCF clients generated from old references, binary serialization, and Remoting need an explicit contract. ConfigurationManager and web.config transforms become the options pattern. Static HttpContext.Current has no place in ASP.NET Core. The context is passed in, or read through IHttpContextAccessor sparingly. App-pool identity assumptions become a managed identity. Blocking synchronous IO that worked at low volume starves the new host under load. Packages that were never built for modern .NET stop the slice until you replace, wrap, or drop them.

Example

```
// The module that used to run for every request. If you miss it,
// tenant resolution disappears and the new API reads the wrong rows.
public sealed class TenantMiddleware(RequestDelegate next)
{
    public async Task InvokeAsync(HttpContext context, ITenantAccessor tenant)
    {
        var id = context.User.FindFirstValue("tid");
        if (!int.TryParse(id, out var tenantId))
        {
            context.Response.StatusCode = StatusCodes.Status401Unauthorized;
            return;
        }
        tenant.Set(tenantId);
        await next(context);
    }
}
```

Classify each blocker as replace, wrap for now, or stop the slice. Publish the list with an owner. A migration without that list becomes a long branch. The list is also how you say no to rewriting the reporting module that is not on the critical path.

What breaks

Porting [Authorize] and assuming the module still ran is how an endpoint ships anonymous. Copying HttpContext.Current into a static helper compiles nowhere and, if forced, captures the wrong request. A package with no modern build blocks the slice. Pretending otherwise blocks the program.

Say this

Modules become middleware I test. Configuration becomes options. The app-pool identity becomes a managed identity. I publish the blocker list with an owner and a decision: replace, wrap, or stop this slice.

Blockers you classify out loud

Blocker	Classification	Example of the decision
System.Web HttpModule	Replace in the slice	Tenant and auth modules become middleware with a test that fails closed.
WCF SOAP a partner still calls	Bridge or leave that host	CoreWCF for that contract only. The rest of the API does not wait.
EDMX	New code model for the slice	Map the migrated aggregate. Compare schema. Do not let EF create a second database.
Web Forms page	Leave it	A page is a product decision, not a class-library port.
web.config appSettings	Options and Key Vault	Secrets leave the file. The rest binds to a validated options class.
Windows service	Worker or Function	A queue or a schedule becomes a Function. A long co-located loop becomes a Worker Service.
COM or the registry	Wrap and isolate	That component may stay on Windows after the API moves. Say so in week one.
Shared SQL login as db_owner	Split identities	The first production slice introduces the runtime role and the migrator.

Example of a wrap with an expiry

```
public interface ILegacyInvoicePdf
{
    byte[] Render(int invoiceId);
}

// The COM renderer stays on a Windows worker.
// The new API calls it over HTTP. The ADR names the removal date.
// New invoices do not take a dependency on COM beyond this interface.
```

Say this

I classify each blocker in the first week. A wrap has an owner and an expiry. A Web Forms page and a partner SOAP endpoint do not block the rest of the API.

EF6 to EF Core without a production surprise

Treat it as a new data layer, not a namespace rename. Recreate the model in code. Map keys, required relationships, delete behaviors, and column types explicitly. Compare the schema EF Core wants with the schema that exists. Do not let it create a parallel database. Port the hot queries first and run them against a realistic dataset, because translation differences change plans. Lazy loading is opt-in and different. Prefer explicit loads and projections. EDMX does not come forward.

During the strangler, EF6 stays in the old process and EF Core stays in the new process. Both may point at the same database only with strict rules: no silent model drift, and one owner of migrations. Spatial types, table-

per-type inheritance, and some complex mappings need an explicit check against the EF Core documentation for the version you are on, not a memory of EF6. Test concurrency tokens and transactions, not only CRUD.

Example

```
// You write the model to match the existing table. You do not scaffold
// and then 'fix it in production'.
modelBuilder.Entity<Order>(b =>
{
    b.ToTable("Orders", "orders");
    b.HasKey(o => o.Id);
    b.Property(o => o.Total).HasColumnType("decimal(18,2)");
    b.Property(o => o.RowVersion).IsRowVersion();
    b.HasQueryFilter(o => o.TenantId == tenant.Id && !o.IsDeleted);
});

// One migration history. The pipeline's identity applies it.
// The Framework app does not call Database.SetInitializer and create tables.
// The new app does not call Database.Migrate at startup.
```

What breaks

Two migration histories on one database drop a column the old app still reads. A rename EF Core inferred from a missing property ships as drop-and-add. Lazy loading that used to fill a grid becomes an N+1, or an empty collection, depending on what you forgot. The in-memory provider will not catch any of this.

Say this

I rebuild the model and compare it to the live schema. One side owns migrations. Hot queries are tested against SQL with realistic stats. I do not call it a port. I call it a new data layer beside the old one.

Authentication, cut over without an anonymous hour

Write down the current mechanism: Windows integrated, forms cookie, OWIN JWT, custom headers, or certificates. Map every audience: browser, partner, and internal job. For a new API, prefer Entra ID and bearer tokens, with policies that match today's roles. Run both authenticators during the overlap if clients cannot all move, and log which scheme succeeded. Test 401 and 403 on every moved route, plus one cross-tenant case. Do not trust a ported [Authorize] until a test fails closed.

Secrets in web.config move to Key Vault references. Certificates get an owner and an expiry alert. A Windows-identity flow that impersonated the user down to SQL is a design change: the user's identity stays in the token, and the app's managed identity connects to SQL. Authorization stays in the API. You do not flow the Windows token into Azure SQL and call that a port.

Example

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o =>
    {
        o.Authority = "https://login.microsoftonline.com/<tenant>/v2.0";
```

```
        o.Audience = "api://orders";
    });

builder.Services.AddAuthorization(o =>
{
    o.AddPolicy("orders.write", p => p.RequireClaim("roles", "orders.write"));
});

// The test that must fail before you celebrate the port:
// no token -> 401
// token without the role -> 403
// token for tenant A requesting tenant B's order id -> 404 or 403, as documented
// and the SQL login is the app identity, not the user's Windows account.
```

What breaks

A route you forgot to authorize ships open. Impersonation to SQL silently stops, or worse, the new app connects as a powerful Windows account you copied into the connection string. A certificate with no expiry alert dies on a Saturday.

Say this

I map every audience, run both schemes only while clients move, and log which one succeeded. 401 and 403 are tests. The user token authorizes. The managed identity connects to SQL.

What you test when the old system has few tests

Start with characterization tests on the HTTP contract of the slice you are moving: status code, the headers that matter, and the JSON shape for the main success and the main failure. Add golden tests around billing calculations and any code that has caused an incident. Add an integration test that hits SQL for the migrated query. Do not stop the migration to unit-test the entire legacy solution. Do not accept a rewrite of a slice with no before-and-after contract check.

For the proxy, test the routing rules: this path goes to the new host, that path stays, and a bad route fails in a way you can see. Put load on the migrated slice. Sync-over-async and a missing index show up there, not in a unit test. Sequence the tests by money and embarrassment. The payment path gets a characterization test before the admin dropdown does.

Example

```
[Fact]
public async Task Create_order_keeps_the_contract()
{
    var response = await client.PostAsJsonAsync("/orders", new
    {
        sku = "AB-1",
        quantity = 2,
        idempotencyKey = "k-1"
    });
```

```
    Assert.Equal(HttpStatusCode.Created, response.StatusCode);
    var body = await response.Content.ReadFromJsonAsync<JsonElement>();
    Assert.Equal("Pending", body.GetProperty("status").GetString());
    Assert.True(response.Headers.Location is not null);
}

// Captured from the old host first. The new host must match.
// A field rename is a contract break, even if the new name is nicer.
```

What breaks

A green suite of new unit tests that never saw the old JSON lets a renamed field break every partner. Skipping the proxy test sends /payments to the new host that does not own it yet. Load left until launch hides the blocking call.

Say this

I characterize the HTTP contract of the slice I am moving, and I hit SQL for the migrated query. I do not boil the ocean. I test payment before I test a dropdown.

How you estimate, and what you refuse to rewrite

Estimate the inventory and the first slice, not the whole estate. The first slice teaches the real cost of auth, EF, and deployment. After it is in production, estimate the rest in slices, with a range. Refuse to rewrite code you do not understand, code with no contract, and code the business is about to replace. Wrap those and put an expiry on the wrap. Refuse a second feature program that lands only on Framework while you migrate. New work goes to the new runtime unless there is a dated exception.

Keep a risk register: shared database, an unknown module, key-person knowledge, a package with no modern build, and a rollback that requires schema compatibility. A single number for an eighty-project solution is fiction. A funded first slice, a risk register, and a cadence is an estimate you can defend.

Example

```
Fund now:
  Week 0 inventory and the blocker list.
  Slice 1: POST and GET /orders, characterization tests, YARP route, pipeline migrate.

Explicitly not in slice 1:
  Rewriting the Web Forms admin.
  Splitting microservices.
  Replacing the reporting warehouse.

Risk register line:
  Shared database. Owner: the tech lead. Mitigation: one migration history,
  expand/contract, route rollback rehearsed before the slice takes traffic.
```

What breaks

An estimate that includes a domain rewrite misses the support date. New features landing on Framework make the inventory stale every sprint. A wrap with no expiry is a permanent legacy host with a nicer name.

Say this

I estimate the first slice and the inventory. I refuse a rewrite of code we do not understand or the business is about to drop. New work goes to .NET 10 unless I write down a dated exception.

Rollback six weeks into a strangler

Routing rollback sends the slice's traffic to the old host. That works only if the old host can still read and write the current schema, which is why you expanded the schema and did not rename or drop in the same release. Data written by the new service must be readable by the old service, or you have a compensating job you have rehearsed. A feature flag can disable a new behavior without a deploy. You do not need to roll back a shared class library if it stayed compatible. Rehearse the route switch in a lower environment. If you cannot describe rollback, the slice is too big. The person on call should not invent it at midnight.

Example

```
Week 1, expand:
  ALTER TABLE orders.Orders ADD Currency char(3) NULL;
  Old app ignores the column. New app writes it.

Week 6, incident on the new host:
  YARP sends POST /orders back to the Framework app.
  The old app still inserts. Currency stays null. That is acceptable
  because the column is nullable and the old app never read it.

Not reversible:
  Week 1 renamed Total to Amount and dropped Total.
  The old INSERT fails. Swap-back is a second incident.

The checklist line next to the migration script:
  'Route /orders returns to the Framework app in one config change.
   Rehearsed on staging on <date>.'
```

What breaks

A drop or rename in the same release as the route change removes the old host as a rollback. Data the new app wrote in a shape the old app rejects is stuck even if the route goes back. An unrehearsed proxy change is a guess at midnight.

Say this

Rollback is a route change plus a schema the old bits can still use. Expand, then contract later. I rehearse it. If I cannot describe it, the slice is too big.

What you adopt during the move, and what you postpone

Adopt the SDK-style project, nullable reference types on new code, generic hosting, the options pattern, structured logging, and EF Core migrations owned by the pipeline. Adopt managed identity if the app is moving to Azure as part of this work. Postpone a microservices split, a new UI framework, and a domain rewrite. The migration's job is a supported runtime, the same business behavior, and a safer deploy.

A cleaner module boundary inside the new host is welcome when it falls out of the slice. A new architecture program disguised as a migration is how the date slips and Framework stays in production past support. .NET 8 and .NET 9 reach end of support in November 2026. New work targets .NET 10 LTS, supported through November 2028. Say the sentence to stakeholders: supported runtime first, structural improvement only where the slice already has to change.

Example

```
In the slice, because the code is already open:
  nullable enabled on the new projects
  options class for TaxUrl, ValidateOnStart
  structured log with trace id
  pipeline migration, runtime SQL role

Not in the slice, written down so the team stops relitigating:
  extracting a pricing microservice
  rewriting the admin UI
  introducing a generic repository 'while we are here'
```

What breaks

A migration that also replatforms to Kubernetes and splits six services misses the support window. Postponing nullable, logging, and the SQL role means you arrive on .NET 10 with the same operational scars.

Say this

Supported runtime first. I adopt hosting, options, logging, pipeline migrations, and managed identity because the slice already touches them. I postpone a new architecture. That sentence is how the date survives.

Day 9 — System design and the lead conversation

Day 9 turns the earlier days into a design you can finish in twenty minutes, and into a story that is yours. The samples below are scaffolds. They show the shape a panel can score. Replace the system, the number, and the decision with your own. A borrowed checkout story collapses on the first follow-up.

The design spine

Write this on the board in the first three minutes. It keeps you from jumping to Kubernetes, and it gives the interviewer a place to interrupt you with a better number.

1. Clarify the user, the success metric, the scale you are assuming, and what is out of scope. State the assumption and let them correct you.
2. Name the API: resources, verbs, sync versus async, status codes.
3. Name the data: the aggregate, the key, the constraint, the index that matches the hot query.
4. Name consistency: what is in one SQL transaction, what is a message, what may lag.
5. Name failure: timeout, retry, duplicate, poison message, and what the user sees.
6. Name operation: logs, the one dashboard, deploy and rollback, identities and SQL roles.
7. Name the tradeoff you refused, and the first slice you would ship.

If they do not give numbers, assume some and say them. For this guide: about 50 requests a second at peak, about 2 million orders a year, p95 under 300 milliseconds for the read, email within a minute. Invite correction. The design changes when the number changes. A lead who designs for a million events a second without being asked overbuilds and runs out of time.

Example of the opening ninety seconds

```
"I am assuming a partner posts orders and retries, a customer reads an order,
and the confirmation email can finish after the response. Peak is about 50
requests a second, a couple of million orders a year, read p95 under 300
milliseconds, email inside a minute. I am not designing a million-event stream.
Tell me if those numbers are wrong.

I will keep one API on App Service, one database, and a Function for email.
I will not split a pricing service in the first slice."
```

Say this

I state the user, the number, and the out-of-scope box before I name a product. Then I take correction. The rest of the design hangs off that number.

Design A — the Order API, worked

Prompt: design the backend for creating an order, viewing it, and emailing a confirmation. Partners retry. Two customers must not lose updates. You are the tech lead. The pieces below are Days 3, 4, 5, 6, and 7 in

one design. Speak them in spine order, not as a tour of Azure.

API

```
POST /orders
  Header Idempotency-Key: <client guid>
  Body: sku, quantity, ship-to. Not Total. Not IsPriority. The server prices.
  201 Created, Location, body     first time
  200 with the stored body        same key, same request hash
  409                             same key, different body
  400                             validation
  401 / 403                       authn / authz
  503                             tax dependency exhausted, if tax is required

GET /orders/{id}
  200 projection, no tracking
  404 in the caller's tenant

GET /orders?cursor=&limit=
  keyset on (CreatedUtc, Id), limit capped

PATCH /orders/{id}
  If-Match or the rowversion in the body
  409 if a concurrent update won
```

Data and consistency

```
One SQL transaction:
  orders.Orders            rowversion column
  orders.OrderLines
  orders.IdempotencyKeys   unique (TenantId, KeyValue), stores response hash
  orders.Outbox            unpublished OrderConfirmed

Unique business key where the domain has one, such as partner reference.
Index (TenantId, CreatedUtc DESC, Id DESC) INCLUDE (Status, Total) for the list.

Outside the transaction:
  the Service Bus publish, done by the relay
  the email, done by the Function

The user does not wait on the mail provider.
Mail status is a column support can see.
```

Failure, operation, and the slice

- Tax, if it is synchronous and required: a timeout inside the request, a small retry only because the tax call is safe to repeat, then 503. If the business would rather accept PendingTax, say that and store the state. Do not invent the policy in the interview without labeling it as a product choice.
- Duplicate email: the consumer dedupes on the message id. That unique key is in the first increment, not a follow-up story.

- Two writers: the rowversion WHERE clause changes zero rows, EF throws, the API returns 409. No blind retry.
- Host: App Service, slots, Always On, affinity off. Function on the isolated worker for email. Three identities: user token, API managed identity in app_orders_runtime, pipeline migrator.
- Dashboard: request latency, SQL duration, outbox age, dead-letter depth. Rollback is a swap, and only while the schema is still compatible.
- First slice: synchronous create, the outbox, and the email consumer. Pricing stays a module behind IPriceService. No pricing microservice.

Close with the week-one risk: a duplicate email from redelivery, which is why the message id is unique in the first ship. If they change the peak to a number that breaks one database, you revisit the read model. You do not revisit it at 50 requests a second.

What breaks

Accepting Total from the client lets the partner set the price. Publishing the email inside the transaction holds locks. A microservice split in the first twenty minutes spends the clock on boxes and never reaches the duplicate or the 409.

Say this

One transaction for the order, the idempotency row, and the outbox. Email after commit, deduped. 409 on a lost update. App Service and a Function, three identities. Pricing stays a module until a second team needs it.

Design B — a nightly file of a million rows, worked

Prompt: a partner drops a CSV in storage every night. You must validate, load, and report failures. The HTTP API must stay responsive. The mistake is to parse a million rows inside the blob trigger or on a request thread.

Flow

```
1. Blob created, or an Event Grid event.
   A function records the file and enqueues one control message.
   It does not parse the file.

2. A splitter reads ranges and enqueues chunk messages:
   { fileId, blobName, rowStart, rowCount }

3. Workers upsert a chunk.
   Each chunk commits with a unique (fileId, rowStart).
   A retry of the same chunk does not double-insert.
   A bad row goes to orders.FileErrors (fileId, lineNumber, reason).
   The file row moves Received -> Processing -> CompletedWithErrors or Failed.

4. The API serves status and the error page from SQL.
   It never parses the file.
```

```
   The partner can fetch an error CSV.

5. Alert if the file is not terminal by the business deadline.
   A poison chunk dead-letters without stopping the other chunks.
   The file state shows the gap.
```

Why not EF per row

```
A million SaveChanges calls will miss the window and hold the API's database busy.
Use a staged table and a set-based merge, or a bulk copy into the stage.

INSERT into orders.FileChunks (FileId, RowStart) is the idempotency key.
The worker's managed identity can read the container and write the staging schema.
It cannot ALTER. The API's identity is not the worker's identity.

Prove chunk retry on a small file before you talk about scale.
Say the plan and the timeout as properties you would check, not as a memorized SKU.
```

What breaks

One invocation that reads the whole file dies at the function timeout and retries from byte zero. SaveChanges per row holds locks and misses the window. A chunk retry without a range key double-inserts. A poison chunk that stops the orchestrator leaves the other 99 chunks unprocessed and the file looking idle.

Say this

The trigger only records the file. Workers own chunks, each chunk is idempotent, bad rows are queryable, and the API only reads status. I prove retry on a small file, and I show the error file the partner can fetch.

Design C — multi-tenant, if they ask

Default to one database. TenantId is on every tenant-owned row. A global query filter applies it. The unique index includes TenantId. The tenant id comes from the token, never from the body. The API authorizes the route, and a test proves tenant A cannot read tenant B's id. The filter is a safety net, not the only control.

A noisy or regulated tenant can move to a separate database. The application resolves the connection by tenant. Migrations must apply to every database, which is an operational cost you name before you offer the option. You do not start with a database per tenant for hundreds of small customers. The migration and the connection story will dominate the team.

Example

```
CREATE UNIQUE INDEX UX_Orders_Tenant_Number
    ON orders.Orders (TenantId, Number);

-- Token claim "tid" = 10.
-- Body field tenantId = 11 is ignored, not trusted.
-- Test: token 10, GET /orders/{id of tenant 11} does not return the row.
```

```
Separate database, only when you can say why:
  a regulated tenant, or a tenant whose volume hurts the others.
  Cost you name first: every migration runs once per database,
  and a missed database is a drift incident.
```

What breaks

Trusting tenant id from the body is a cross-tenant write. A unique index on Number alone collides across tenants or, worse, leaks that a number exists. A database per small tenant multiplies every operational mistake by the tenant count.

Say this

One database, TenantId from the token, a filter, a unique index that includes the tenant, and a test that crosses tenants. A separate database is a named exception with a migration cost, not the default.

How a lead story is shaped

Use situation, task, action, result, and a last line: what the team does now. Keep it under two minutes. I is for your decision. We is for the work the team did. Panels listen for whether you actually decided something. The three stories in the roadmap are scaffolds. Below, each one is annotated so you can hear what is doing the work. Then you throw the facts away and write yours.

Scaffold — production incident

Situation: a release went out at 11:00 and by 11:20 the order GET p95 was above four seconds. Task: you were the tech lead on call and had to choose rollback or fix-forward. Action: you checked the deploy marker against the latency chart, saw SQL dependency time jump, and swapped back within fifteen minutes because the migration had only added a nullable column. Then you looked at Query Store. A new screen was scanning Orders. The team added a covering index in a reviewed script and shipped it that afternoon. Result: latency returned to about 200 milliseconds after the swap, and the index kept the new screen. The team rule now is that a list-endpoint PR includes the filter, the index, and a sample plan, and rollback is pre-written on the release ticket.

- The decision is the swap, and the minute you chose it. Without the minute, it is a war story.
- The number is the p95 before and after. A panel can score a number. It cannot score 'it was slow'.
- The safety condition is the nullable column. That is why swap-back was allowed. Say it.
- The lasting line is the PR rule. A lead changes the next review, not only tonight's graph.

What breaks

Memorizing this checkout incident as your career fails when they ask which index and which table. Replace every fact. If you do not have a p95, use the number you did have: error count, tickets, or how long the swap took.

Scaffold — disagreement

Situation: a senior developer wanted a generic repository and a new microservice for pricing before a release. Task: you needed a data-access approach the team could review, and a date you could hit. Action: you asked what change you were protecting. You had one query shape and one team. You proposed pricing as a module behind IPriceService, and no generic repository, so the SQL stayed visible. You wrote a one-page note and asked them to own the module boundary. Result: you shipped on time, the query stayed tunable, and they kept real ownership. You would split pricing later if a second team needed to release it.

- You disagreed with a person, not with 'the business'. Name the proposal.
- The question 'what change are we protecting?' is the technique. Use it on your own story.
- The relationship cost is in the story: they kept ownership. A lead who only wins the argument sounds expensive to work with.
- The consequence is a date you hit and a SQL statement you could still see.

Scaffold — growing someone

Situation: a developer with two years of experience was copying controllers and calling SaveChanges from actions. Task: the next feature had to land in a shape the team could maintain. Action: you paired on one endpoint, moved the use case into a handler, and left them a checklist: cancellation token, a projection for the read, a test for the failure path, and a status code. You reviewed the rule, not the formatting. Result: they owned the returns API two months later and caught an over-posting bug in someone else's PR. The checklist is in the repo.

- The action is a checklist another person can apply. Heroic pairing with no artifact does not scale.
- The result is what they own now, plus a bug they caught without you.
- I reviewed the rule. We shipped the endpoint. Keep those pronouns honest.

> **Say this**
>
> My story has a decision, a number, and a rule the team still follows. I is the decision. The sample in the guide is the shape. The facts are mine.

A missed date

Pick a real date. Say the original commitment, the new risk you learned, and the day you told stakeholders, which should be early. Say what you cut: a report, an edge case, a non-critical admin screen. Say what you refused to cut: auth, a migration rollback, or the payment test. Say the new date you hit and the planning change, such as estimating the first slice only. Avoid a story where you worked a weekend and changed nothing about planning.

Example of the shape, not your career

```
Commitment: order slice in production on 12 March.
Learned on 20 February: the partner certificate path was a second audience
we had not inventoried.
Told stakeholders the same day.
```

```
Cut: the admin export.
Refused to cut: the 401 test and the route rollback rehearsal.
New date: 26 March. We hit it.
Planning change: the next slice starts with the audience list, not with code.
```

What breaks

A story about quietly absorbing the slip is an individual-contributor story. A story that blames the stakeholder and cuts the payment test is a lead story in the wrong direction.

Say this

I re-planned in public, early. I cut scope that was not auth, data, or rollback. I changed how the next slice is estimated.

Technical debt

Name the debt in user or operator language: this list endpoint will time out at the next tenant's volume, or this Framework host is out of support in November 2026. Attach a cost to leave it and a cost to fix it. Put a slice of the fix in the same increment as a nearby feature when the code is already open. Spend a reserve on the item with the nearest incident, not on the rewrite you find interesting. Do not hide a migration inside an unrelated story. Also delete debt. A pattern you no longer need is a ticket with an owner.

Example

```
Debt: GET /orders has no keyset and no covering index.
Cost to leave: the next large tenant pushes p95 through the 300 ms budget
and shows up as a support ticket, the way the last scan did.
Smallest fix: the index and the cursor, in the PR that is already adding
a column to that list.
Not the fix: a read-replica program, this quarter.
Owner and date go on the board. Vague passion about clean code does not.
```

Say this

I name the incident the debt implies and the smallest fix. I attach it to work that is already open. I do not spend the reserve on the rewrite I find interesting.

Review without becoming the bottleneck

Publish the rules so people can apply them without you: cancellation, authorization, SQL shape, secrets, tests for the money path, and migration safety. Review the contract and the risk. Let the formatter own the braces. Be available at agreed times so reviews do not sit overnight. Ask the author to walk you through a design before a thousand-line PR exists. Two approvals for auth and for schema. A junior's first reviews are paired so the comments teach. If every PR waits for you, you have failed. Pick a second reviewer and a written rule that removes you from the middle.

Example

```
The comment that teaches, from the bad Create action on Day 3:

"We do not bind IsAdmin or TenantId from the body. Add a command type
that contains only sku and quantity. The tenant comes from the token.
Please apply the same split on the returns endpoint in this PR."

Not: "nit: brace on the next line."
```

Say this

I publish the rule, I review the contract and the risk, and I make sure a second person can approve the ordinary PR. If everything waits for me, the review system has failed.

Disagreeing with your manager's technical direction

Restate their goal until they agree you understood it. Put your alternative in writing with the risk, the date, and the operational cost. Distinguish a reversible choice from a one-way door. On a reversible choice, disagree, commit, and watch the metric you named. On a one-way door, such as a shared database across a boundary you cannot undo, or a production login as db_owner, say that clearly and offer a smaller experiment. Do not relitigate in the team channel after the decision. If the decision is unsafe for customers or staff, escalate with the risk written down.

Example

```
Goal I heard: ship the partner API this quarter.
Direction: one shared SQL login in db_owner so the new service can move faster.
That is a one-way door. The login will still be there after the quarter,
and so will every table it can drop.

Alternative: app_orders_runtime this week, migrator in the pipeline,
two extra days on the identity. I will not relitigate the choice of App Service
versus Functions for the email worker. That one is reversible. We can watch
the mail latency and move it.
```

What breaks

Calling every disagreement a one-way door makes the phrase useless. Relitigating a reversible choice in public after you committed trains the team to ignore the decision. Staying silent on db_owner because the date is close is how the login survives for years.

Say this

I restate the goal, I write the risk, and I separate a reversible choice from a one-way door. I use that language once, on a real example, and then I stop.

Questions you ask them

A lead who asks about rollback, identity, and on-call sounds like they intend to live with the system. Ask two, not six, at the end of a round. Pick the two that match what you heard.

- What is the first slice you would want me to lead in the first 90 days?
- Which systems are still on .NET Framework, and is there a funded plan before support ends?
- How does a change reach production, and who can roll it back at 21:00?
- What does on-call look like, and what woke the team up last month?
- Where should a tech lead spend time here: design, review, people, or still a large share of coding?
- How are SQL changes reviewed, and what rights does the application identity have today?

Example

```
"You mentioned the Framework billing API.
Is there a funded slice onto a supported runtime, and who can roll a release
back at night if the new path mis-prices?"

Two questions. Then stop talking.
```

Say this

I ask how a change reaches production and what the app's SQL login can do. I ask two questions, tied to something they already said.

Day 10 — Mock day and compression

Day 10 adds no new theory. The roadmap's job today is speaking: ninety seconds, rapid fire, a blank-page design, your own stories. This chapter is the check. Each rapid-fire line has a miniature example and a pointer to the full section. If the miniature feels thin, go back. Do not open a new topic because you are nervous.

The ninety-second shape

Sentence one is the decision. Sentence two is how it works in the Orders example. Sentence three is the tradeoff or the person who feels it. Then stop. Silence is fine. If you hear yourself listing pattern names, return to the problem.

Example, spoken against a timer

```
Question: how do you keep a retried POST from creating two orders?

"The client sends an Idempotency-Key, and I store it uniquely with the
response in the same transaction as the order. A retry with the same key
and the same body returns the stored response. The same key with a different
body is a 409. I still keep a unique business key, because an idempotency
header is not a data rule. The tradeoff is retention: I keep the key for as
long as the client retries, not forever."

That is the whole answer. The outbox, the email, and the rowversion wait
until they ask.
```

- Round 1: eight questions from the roadmap, ninety seconds, one follow-up, scored.
- Round 2: the miniatures below. A hesitation means the short answer is not yours yet.
- Round 3: Design A from a blank page in twenty minutes, then a two-minute close.
- Round 4: your incident story and your migration plan, back to back. The Day 9 scaffolds are not your script.

> **Say this**
> Decision, example, tradeoff, stop. I practice out loud. I do not add a chapter the night before.

Rapid fire, each with a miniature

Read the miniature, cover it, and say the one-sentence answer. The pointer is where the full example lives if you miss.

401 versus 403

401 means we do not know you. 403 means we know you and you may not do this. A missing bearer token on POST /orders is 401. A valid token without orders.write is 403. Full treatment: Day 3.

```
no Authorization header        -> 401
valid token, missing role      -> 403
valid token, other tenant's id -> 404 or 403, and you have written which
```

PUT versus PATCH versus POST

PUT replaces the resource. PATCH changes a part. POST creates, or starts work that is not a simple replace. Full treatment: Day 3.

```
PUT   /orders/10   body is the whole order you want stored
PATCH /orders/10   body is { "status": "Cancelled" }
POST  /orders      creates; a retry needs an idempotency key
```

Transient, scoped, singleton

Per resolve, per request, per process. DbContext is scoped. A singleton must not take a scoped service in its constructor. Full treatment: Day 3.

```
AddDbContext<AppDbContext>();          // scoped
AddSingleton<IClock, SystemClock>();  // no DbContext in this constructor
// A hosted service creates a scope per unit of work.
```

Why not .Result

It blocks a thread-pool thread and can starve the API under load. Await, and pass a CancellationToken. Full treatment: Day 1.

```
var orders = db.Orders.ToListAsync(ct).Result;   // review reject
var orders = await db.Orders.ToListAsync(ct);
```

IEnumerable versus IQueryable

IEnumerable runs in memory. IQueryable can become SQL. Materialize at the boundary. Full treatment: Day 1.

```
IEnumerable<Order> q = db.Orders.Where(o => o.TenantId == id);
q = q.Where(o => o.Status == "Open");   // this Where is LINQ to Objects
// The SQL already returned every order for the tenant.
```

AsNoTracking

Use it on reads so EF does not snapshot entities you will not update. Full treatment: Day 4.

```
await db.Orders.AsNoTracking()
    .Where(o => o.Id == id && o.TenantId == tenantId)
    .Select(o => new OrderListItem(o.Id, o.Number, o.Total))
    .SingleOrDefaultAsync(ct);
```

N+1

One query for the parents and one query per child, usually a loop or a lazy load. Replace it with one projection or a controlled include. Full treatment: Day 4.

```
foreach (var order in orders)
    order.Lines = await db.Lines.Where(l => l.OrderId == order.Id).ToListAsync(ct);
// One round trip instead: project the page, include lines only if the UI needs them.
```

rowversion

A concurrency token. The update matches the old value. Zero rows means 409, not a silent overwrite. Full treatment: Day 4.

```
UPDATE orders.Orders
SET Status = @Status, RowVersion = <generated>
WHERE Id = @Id AND RowVersion = @Original;
-- @@ROWCOUNT = 0 -> DbUpdateConcurrencyException -> 409
```

Clustered versus nonclustered

Clustered orders the table. Nonclustered is a secondary key path. Include columns to avoid lookups. Full treatment: Day 5.

```
PRIMARY KEY CLUSTERED (Id)
INDEX (TenantId, CreatedUtc DESC, Id DESC) INCLUDE (Status, Total)
```

SARGable

The column stands alone so an index can seek. Do not wrap the column in a function. Full treatment: Day 5.

```
WHERE YEAR(CreatedUtc) = 2026                          -- scan
WHERE CreatedUtc >= '2026-01-01' AND CreatedUtc < '2027-01-01'  -- seek
```

Deadlock 1205

A lock cycle. SQL kills a victim. Retry if the work is safe, then remove the cycle with one lock order, an index, and a shorter transaction. Full treatment: Day 5.

```
Session A: update Orders 10, then Payments 88
Session B: update Payments 88, then Orders 10
-- Pick one order. Do not hold either transaction across an HTTP call.
```

RCSI

Readers use row versions and do not block writers the way shared locks do. It does not replace a concurrency token. Full treatment: Day 5.

```
ALTER DATABASE CURRENT SET READ_COMMITTED_SNAPSHOT ON;
-- Two sessions can still both read a balance of 10 and both write 11.
```

```
-- The fix is an atomic UPDATE or a rowversion, not the isolation flag.
```

db_owner for the app

No. A custom runtime role for DML. A separate identity for migrations. Full treatment: Day 5.

```
app_orders_runtime  : SELECT, INSERT, UPDATE, DELETE, EXECUTE on schema orders
app_orders_migrator : ALTER on schema orders, pipeline only
```

Idempotency key

A client key stored uniquely with the first response, so a retried POST does not create a second order. Full treatment: Day 3 and Design A.

```
UNIQUE (TenantId, KeyValue)
same key, same body  -> stored 201
same key, other body -> 409
```

Outbox

The business row and the message row commit together. A relay publishes. Consumers dedupe. Full treatment: Day 7.

```
BEGIN TRAN  insert Orders; insert Outbox; COMMIT
-- relay sends after. A crash before the send leaves a visible unpublished row.
```

Peek lock

Service Bus hides the message until you complete it. Abandon or lock expiry makes it visible again. Too many failures dead-letter it. Full treatment: Day 6.

```
handle -> SQL commit -> Complete
throw  -> Abandon -> retry until MaxDeliveryCount -> dead-letter
```

Cold start

A scaled-to-zero function pays startup latency on the next call. Buy a warm plan if a user is waiting. Do not memorize a millisecond. Full treatment: Day 6.

```
Email that can take a minute: Consumption is a real option.
Checkout the user is staring at: not a scale-to-zero plan.
```

Deployment slot

Stage and warm the new bits, swap, keep the old bits for swap-back. The schema must still suit both. Full treatment: Day 6 and Day 4.

```
add nullable column -> deploy slot -> ready check -> swap
rename-and-drop in the same release -> swap-back is a lie
```

Managed identity

The Azure resource authenticates without a password. Grant it Key Vault and a SQL user in a least-privilege role. Full treatment: Day 6.

```
Authentication=Active Directory Managed Identity
CREATE USER [id-api-prod] FROM EXTERNAL PROVIDER;
ALTER ROLE app_orders_runtime ADD MEMBER [id-api-prod];
```

Strangler fig

Route one slice at a time to the new host. Delete the old slice only when traffic, logs, and rollback are proven. Full treatment: Day 8.

```
POST /orders -> net10
GET  /catalog -> Framework, until its own slice
metric: percent of order traffic, not percent of projects
```

Generic repository

DbContext is already a unit of work. Prefer a focused store per aggregate so the SQL stays visible. Full treatment: Day 2.

```
interface IOrderStore { Task<Order?> GetAsync(int id, CancellationToken ct); Task SaveAsync(Order
order, CancellationToken ct); }
// Not IRepository<T> with Get, Add, and a predicate that hides Include and ExecuteUpdate.
```

Mediator

A dispatcher to one handler, useful when pipeline behaviors earn it. Handlers do not call handlers. Full treatment: Day 2.

```
CreateOrder -> CreateOrderHandler
validation and the transaction are pipeline behaviors
the handler does not send GetOrder through the mediator
```

Circuit breaker

Fail fast when a dependency is sick, then probe. Retries need a budget or they amplify the outage. Full treatment: Day 3.

```
tax client: timeout, two jittered retries, breaker
open breaker -> 503 now, not a 30-second hang
do not retry a charge unless the idempotency story is solid
```

Sync versus async integration

Sync when the user needs the answer now and you accept the dependency. Async when the work can finish after the response. Full treatment: Day 7.

```
tax required to price  -> sync, then 503 if it is down
email confirmation     -> outbox, 201 immediately
```

Database per service

The service owns its writes. Cross-service joins become an API, a copied snapshot, or a reporting store. Full treatment: Day 7.

```
orders.CustomerSnapshot filled by CustomerChanged
-- not a SQL login from Orders onto customers.Customers
```

Startup Migrate

Unsafe with many instances. Apply a reviewed SQL script from the pipeline before traffic moves. Full treatment: Day 4.

```
// not this
app.Services.GetRequiredService<AppDbContext>().Database.Migrate();
// this: pipeline, migrator identity, then the slot swap
```

ConfigureAwait(false)

Important in a library that must not resume on a captured context. ASP.NET Core application code has no sync context to return to. Blocking calls still hurt. Full treatment: Day 1.

```
// library code that may be called from a UI or older ASP.NET
await stream.ReadAsync(buffer, ct).ConfigureAwait(false);

// a controller: just await, and do not call .Result
```

Record versus class

A record for a value-shaped contract. A class for an entity with identity that EF tracks. Full treatment: Day 1.

```
sealed record OrderCreated(int OrderId, DateTimeOffset OccurredUtc);
sealed class Order { public int Id { get; set; } }
```

ProblemDetails

A consistent error body with status, title, and a trace id. Clients and logs share it. Full treatment: Day 3.

```
{ "status": 409, "title": "Conflict", "traceId": "441" }
validation adds an errors dictionary. Do not invent a new shape per endpoint.
```

Keyset versus offset

Keyset seeks the next row from the last key. Offset discards skipped rows and gets worse on deep pages. Full treatment: Day 5.

```
WHERE CreatedUtc < @CursorUtc OR (CreatedUtc = @CursorUtc AND Id < @CursorId)
ORDER BY CreatedUtc DESC, Id DESC
```

.NET support

.NET 10 is the LTS to target, supported into November 2028. .NET 8 and .NET 9 reach end of support in November 2026. Full treatment: Day 8.

```
new work: net10.0
a Framework host still in production past that window is a dated risk,
not a backlog theme
```

Where the mock prompts live

Pull eight at random, as the roadmap says. This table is only a map so a miss sends you to a chapter instead of to a new Google search.

Prompt	Open
Tell me about yourself as a backend lead.	The positioning line in the roadmap front matter. Decisions, cost, rollback.
Explain async to someone who used .Result.	Day 1, async. Show the blocked call and the token.
Design POST /orders so partners can retry.	Day 9 Design A. Idempotency key, unique constraint, stored response.
Your list endpoint is slow. Walk the first hour.	Day 6 incident sequence, then Day 4 N+1 and Day 5 plans.
Would you use a generic repository?	Day 2. DbContext is the unit of work. Focused store, or no store.
Two updates hit one row.	Day 4 rowversion. 409. No blind retry.
What SQL role does the application have?	Day 5. Runtime role versus migrator. Not db_owner.
App Service or Functions?	Day 6. API versus email.
What is a dead-letter queue for?	Day 6 peek lock. Poison payload, alert, replay.
Should this system be microservices?	Day 7. Module first. Name the data owner and the failure.
Migrate a Framework Web API to .NET 10.	Day 8. One slice, traffic percent, expand/contract rollback.
A production incident.	Your story, Day 9 shape. Decision minute, number, team rule.
A disagreement on design.	Your story. What change were you protecting?
What do you look for in a pull request?	Day 3 review, and the bad Create action.
What do you want to know about on-call and release?	Day 9 questions. Ask two.

How to score, and when to stop

Score each practice answer from 0 to 2 on three lines: correct, tradeoff, consequence. A 6 is a hire signal on that question. A correct answer with no consequence still sounds like year four. Any row under 4 gets one

more spoken pass the next morning. It does not get a new chapter tonight.

Example of a score

```
Question: startup Migrate.
Correct 2:    many instances race, and a bad script takes the fleet down at boot.
Tradeoff 2:   the pipeline is slower than a startup call and needs a migrator login.
Consequence 2: a failed boot is an outage, and db_owner on the app is the second incident.
6/6. Move on.

Question: RCSI.
Correct 2, tradeoff 0, consequence 0.
You defined it and stopped. Tomorrow morning, add: version store cost,
and it does not fix a lost update. That is the missing 4 points.
```

Depth on orders, SQL, and one migration story beats a thin pass over every pattern name. The one-page sheet you rewrite by hand is the one you may read the night before: positioning line, pipeline order, lifetimes, the status codes you mix up, the transaction boundary, the index for your hottest list, the runtime role versus the migrator, peek lock through dead-letter, the strangler metric, the incident in six lines, and two questions for them.

Say this

I still want to design the backend and read the SQL. The difference in this role is that I also own the boundary, the release, the permissions, and the way the team reviews the next change. I will leave you with a decision, the cost of that decision, and how we roll it back.

After the interview

Within an hour, write down every question they asked. Mark the ones where you listed tools and forgot the consequence. Those are the only topics to repair before the next round. Send a short thank-you that mentions one specific thread, such as their migration or their on-call model, and the judgment you would bring to it. Two paragraphs. Do not attach this guide.

The last check

You already have eight years. The panel is deciding whether they can hand you a production API, a SQL database, a release, and a less experienced developer. Speak in decisions, costs, and rollbacks. Use your own incidents. The examples in this guide are the evidence you understand the mechanism. They are not the story of your career.